

Fachhochschule Aachen
Campus Jülich

Fachbereich 9 Medizintechnik und Technomathematik

*Integration von
OpenGL-Visualisierungstechniken
in GPU-Anwendungen*

Masterarbeit des Studienganges Technomathematik

Jülich, August 2012

Anna Jakobs
Peter-Grünberg-Institut / Jülich Centre of Neutron Science
Technische und administrative Infrastruktur

Diese Arbeit wurde betreut von:

Prof. Dr. rer. nat. Martin Reißel
Fachhochschule Aachen

Josef Heinen
PGI/JCNS

Eidesstattliche Erklärung zur Masterarbeit

Diese Arbeit ist von mir selbständig angefertigt und verfasst. Es sind keine anderen als die angegebenen Quellen und Hilfsmittel benutzt worden.

Jülich, den

(Unterschrift)

Abstract

Wissenschaftler am Peter Grünberg Institut / Jülich Centre for Neutron Science untersuchen in Experimenten und Simulationen Form und Dynamik von Materialien wie Polymeren, Zusammenlagerungen großer Moleküle und biologischen Zellen sowie die elektronischen Eigenschaften von Festkörpern. So werden am Institut Quanten-Theorie der Materialien unter anderem Simulationen zur Spindynamik (SD) in magnetischen Nanostrukturen für Anwendungen zukünftiger Datenspeicher durchgeführt. Für das Verständnis der in diesem Zusammenhang anfallenden Ergebnisse ist eine Visualisierung der Magnetisierungsvektoren in verschiedenen Darstellungsmodi oft hilfreich.

Im Rahmen dieser Masterarbeit ist zunächst eine CUDA-basierte GPU-Anwendung in ein OpenCL-basiertes Programm zu überführen, welches den rechenintensiven Teil der Simulation - ein stochastisches Runge-Kutta Verfahren - sowohl auf GPUs als auch Mehrkern-CPU's ausführen kann. Darüber hinaus ist eine Methode zu entwickeln, welche den gleichzeitigen Zugriff einer OpenGL-basierten Visualisierungs-Anwendung auf die aktuell berechneten Ergebnisse der SD-Simulation erlaubt. Dieses Verfahren ist als generisches API umzusetzen, welches auch in anderen GPU-Programmen zum Einsatz kommen kann.

Weiterer Bestandteil der Arbeit ist neben der Implementierung verschiedener effizienter Darstellungsverfahren die Implementierung eines OpenCL-basierten FFT-Algorithmus für nicht-äquidistante Daten. In diesem Zusammenhang soll unter anderem eine Visualisierung der Magnetisierungsvektoren von dreidimensionalen Nanostrukturen ermöglicht werden, bei denen auch Schnitte durch diese Strukturen betrachtet werden können. Des weiteren ist vorzusehen, in thermisch angeregten magnetischen Strukturen eine Echtzeit-Darstellung der Fouriertransformierten zu analysieren, um magnetische Phasenübergänge zu erkennen.

Inhaltsverzeichnis

1. Motivation	5
2. Spindynamics	7
3. OpenCL	11
4. OpenGL	15
4.1. Grundlagen	15
4.2. OpenGL Utility Toolkit	16
4.3. GL framework	17
4.4. Vertex Buffer Objects	17
5. Fouriertransformation	21
6. Entwicklungsumgebung	23
7. Programmierung	25
7.1. OpenCL	25
7.2. Visualisierung	27
7.2.1. VBO	27
7.2.2. GLFW	31
7.2.3. Benutzerinteraktion	33
7.3. FFT	35
7.4. Kompilierung und Ausführung	37
7.5. Programmüberblick	40
8. Ergebnisse und Zusammenfassung	41
9. Ausblick	45
A. Prozessordaten	47
B. Lösung der SDE	48
B.1. Translation of the stochastic differential equation	48
B.2. Runge-Kutta Integration Scheme	49
Literatur	51

1. Motivation

Zwei Aspekte sind im Bereich der wissenschaftlichen Programmierung besonders wichtig: eine möglichst schnelle Durchführung komplizierter und aufwändiger Berechnungen und eine anschauliche Visualisierung der Ergebnisse. Genau diese beiden Punkte waren auch die Motivation zu dieser Masterarbeit.

Ein bereits existierendes Fortran-Programm zu Spindynamics enthielt einige aufwändige Berechnungen, unter anderem ein Runge-Kutta-Verfahren. Diese Berechnungen waren bereits in CUDATM (Compute Unified Device Architecture) überführt worden, einer von NVIDIA entwickelten API, mit welcher der Programmierer Programmteile implementieren kann, die anschließend auf der Grafikkarte parallel ausgeführt werden können. Dies führt oft zu einer deutlichen Beschleunigung der Berechnung. Allerdings hat CUDATM einige Nachteile, es ist beispielsweise nur auf NVIDIA-Karten einsetzbar. Da es aber im JCNS (Jülich Centre of Neutron Science) des Forschungszentrums Jülich, wie auch in den meisten anderen Instituten, nicht nur NVIDIA-Grafikkarten gibt, sondern beispielsweise auch solche der Firma AMD, ist CUDATM nur bedingt geeignet. Wissenschaftliche Programme beziehungsweise Programme im allgemeinen sollten möglichst portabel und vielseitig einsetzbar sein, das ist bei Bindung an eine bestimmte Grafikkarte natürlich nicht der Fall. Daher wurde OpenCL (**O**pen **C**omputing **L**anguage) als neue API für die Berechnungen auf der Grafikkarte gewählt. OpenCL wurde bereits im Rahmen einer Bachelorarbeit[1] getestet und für sehr nützlich befunden.

Zusätzlich zu dieser Änderung sollte eine Visualisierung in das Programm eingebaut werden. Es werden in diesem Programm sehr viele Iterationen von immer denselben Berechnungen durchgeführt. Die Anforderung war eine Live-Visualisierung, das bedeutet, dass nach jeder Iteration die soeben neu berechneten Ergebnisse grafisch dargestellt werden sollten. Dies ermöglicht es den Wissenschaftlern, die Berechnungen in Echtzeit zu verfolgen, was die Ergebnisanalyse und die Beobachtung der Simulation deutlich komfortabler und einfacher macht.

Für die Visualisierung sollte aufgrund ihrer weiten Verbreitung und sehr guten Ausgereiftheit die Programmierschnittstelle OpenGL verwendet werden. Wie sich im Laufe dieser Masterarbeit herausstellte, bietet OpenGL in diesem speziellen Fall noch einen zusätzlichen, sehr wichtigen Vorteil, der im Kapitel 7 Programmierung näher erläutert wird.

2. Spindynamics

Das Programm, das für diese Masterarbeit umgeschrieben wurde, ist ein Programm zur Berechnung der zeitlichen Entwicklung atomarer Spins eines physikalischen Systems. Um Berechnungen bei endlicher Temperatur durchzuführen kann ein Wärmebad hinzugefügt werden. Der Spin eines Teilchens ist eine seiner quantenmechanischen Eigenschaften. Die Spins der einzelnen Teilchen in einem Molekül interagieren, wodurch sich ihre Richtungen ändern können. Genau diese zeitliche Entwicklung wollen Wissenschaftler berechnen und simulieren. Ein wichtiger Bereich, in dem Spindynamics eine große Rolle spielt, ist die Erforschung und Entwicklung neuer Arten von Massenspeichern, da die Grenzen der magnetischen Speichermedien, wie bei den heutzutage verwendeten Festplatten, erreicht sind. So können z.B. neue Wege gefunden werden, um die Schaltzeit von Bits in Festplatten zu beschleunigen.

Dieses Kapitel soll grob die physikalischen Grundlagen hinter den Berechnungen erläutern, die im Spindynamicsprogramm durchgeführt werden. Gegeben ist eine Anordnung von Atomen. Der Spin der Elektronen in einem Atom kann in guter Näherung als kollinear angenommen werden. So kann der Spin eines Atoms i in klassischer Näherung durch einen dreidimensionalen Vektor $\vec{S}_i$ beschrieben werden.

Die Interaktion zwischen atomaren Spins kann wie seine elektronische Eigenschaft durch Verwendung eines Modelhamiltonoperators beschrieben werden. Der historisch relevante ist der Heisenberg Hamiltonoperator:

$$\mathcal{H} = \sum_{i,j} J_{ij} \cdot \vec{S}_i \cdot \vec{S}_j \quad (1)$$

Die Konstante J_{ij} beschreibt die Kopplung zwischen dem Spin des Atoms i und dem des Atoms j . Bei einer negativen Kopplungskonstante streben die beiden Spins danach, sich parallel zueinander auszurichten, bei einer positiven Konstante anti-parallel.

Zur exakten Beschreibung eines physikalischen Systems reicht dies allerdings oft nicht aus und es wird ein erweitertes Heisenbergmodell verwendet.

Zunächst soll die Temperatur bei der Berechnung außer Acht gelassen werden. Dann liefert eine klassische Näherung der quantenmechanischen Bewegungsgleichung folgendes gekoppeltes System von Differentialgleichungen:

$$\frac{\partial \vec{S}_i}{\partial t} = \vec{H}_{i,eff} \times \vec{S}_i \quad (2)$$

$\vec{H}_{i,eff}$ bezeichnet man als das effektive Feld. Dieses beschreibt den Einfluss aller anderen Spins $j \neq i$ auf den Spin i . Formal hat das Feld die Form eines externen magnetischen

Feldes und wird deshalb als lokales effektives Feld bezeichnet. Es berechnet sich, in dem man die Energie eines jeden Spins nach dem Spin selbst ableitet:

$$\vec{H}_{eff} = \frac{\partial \mathcal{H}}{\partial \vec{S}_i} \quad (3)$$

Die Änderung des Spins mit der Zeit, also das Kreuzprodukt von $\vec{H}_{eff}$ und $\vec{S}$, ist ein Vektor, der orthogonal sowohl zu dem Spin als auch zum effektiven Feld steht. In diese Richtung bewegt sich der Spin. Der Betrag des Spins bleibt hierbei konstant (dies lässt sich einfach durch die Skalarmultiplikation beider Seiten mit $\frac{\vec{S}_i}{2}$ zeigen).

Grafisch lässt sich dies wie folgt darstellen:

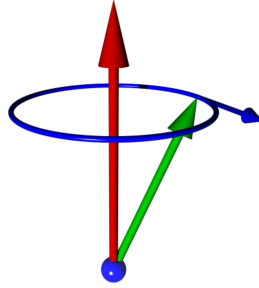


Abbildung 1: Bewegung des Spins (grün) um einen Vektor des effektiven Feldes (rot)[2]

Würde die Bewegung nur über oben stehende Gleichung (2) berechnet werden, würde das nicht dem eigentlichen physikalisch korrekten Verhalten der Spins entsprechen. In der Praxis würde ein einzelner Spin, gerade bei niedrigen Temperaturen, aufgrund von Dämpfungseffekten nach einiger Zeit in das Energieminimum relaxieren und nicht endlos präzedieren, wie das nach obiger Gleichung der Fall wäre. Es wird daher ein zusätzlicher Term in die Gleichung eingefügt, der diese Dämpfung beschreibt:

$$\frac{\partial \vec{S}_i}{\partial t} = \vec{H}_{eff} \times \vec{S}_i + \lambda \cdot (\vec{H}_{eff} \times \vec{S}_i) \times \vec{S}_i \quad (4)$$

Der Dämpfungsvektor ist orthogonal zum Spin und zeigt in Richtung des effektiven Feldes, wie Abbildung 2 zeigt. Die positive Konstante λ gibt hierbei die Stärke der Dämpfung an.

Zusätzlich zum Dämpfungsterm wird noch ein weiterer Term in der Gleichung verwendet, um thermische Anregungen der Spins zu simulieren. Dies geschieht über ein stochasti-

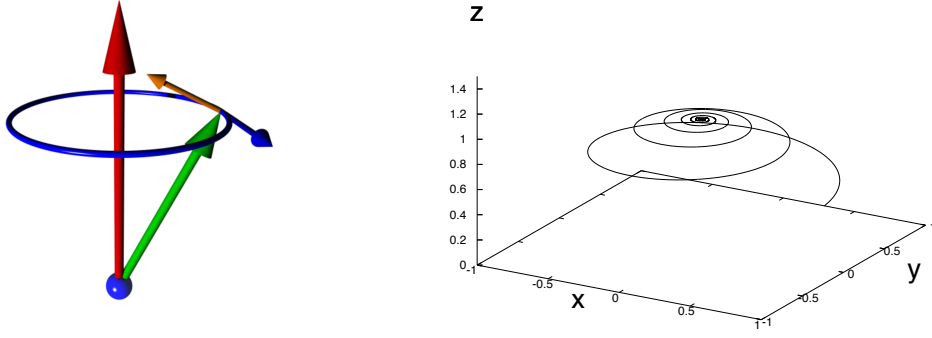


Abbildung 2: links: Dämpfung der Spinbewegung durch Dämpfungsterm (orange),
rechts: Beispiel für Bewegungsverlauf eines gedämpften Spins[2]

sches Feld $\vec{f}$. An dieses Feld gibt es einige Anforderungen, die es erfüllen muss. Zunächst muss für den Mittelwert der Verteilung gelten:

$$\langle f_i^\alpha \rangle = 0 \quad (5)$$

α ist hier eine der Richtungen x, y oder z, das heißt das System verschiebt sich nicht in eine Richtung. Weiterhin soll gelten, dass die Komponenten des stochastischen Feldes unkorreliert sind, sowohl für jeden Spin als auch für die einzelnen Komponenten des Spins und ebenso über die Zeit. Dies wird durch folgende Gleichung beschrieben:

$$\langle f_i^\alpha(t), f_j^\beta(t') \rangle = \delta_{ij} \delta_{\alpha\beta} \delta(t - t') \epsilon^2 \quad (6)$$

Die Standardabweichung, welche die Stärke der fluktuierenden Kraft beschreibt, hängt nach dem Fluktuations-Dissipations-Theorem[3] von der Temperatur und der Stärke der Dämpfung ab:

$$\epsilon^2 = 2\lambda k_B T \quad (7)$$

So wird sichergestellt, dass die Energie des Spinsystems einer Boltzmann-Verteilung folgt ($P(E) \sim \exp(-E/k_B T)$)[2]. λ ist die bereits erwähnte Dämpfungskonstante, T die Temperatur und k_B die Boltzmann-Konstante. Man spricht bei diesem fluktuierenden Feld auch vom „weißen Rauschen“.

Baut man das stochastische Feld $\vec{f}$ in die Gleichung ein, erhält man die sogenannte Landau-Lifschitz-Gleichung[4]:

$$\frac{\partial \vec{S}_i}{\partial t} = \vec{H}_{i,eff} \times \vec{S}_i + \lambda \cdot (\vec{H}_{i,eff} \times \vec{S}_i) \times \vec{S}_i + \vec{f}_i \times \vec{S}_i \quad (8)$$

Das Kreuzprodukt des fluktuierenden Feldes mit den Spins ergibt den stochastischen Term. Dieser steht immer senkrecht zum Spin, so dass dessen Länge erhalten bleibt. Ein Beispiel eines solchen stochastischen Terms und sein Einfluss werden in Abbildung 3 dargestellt:

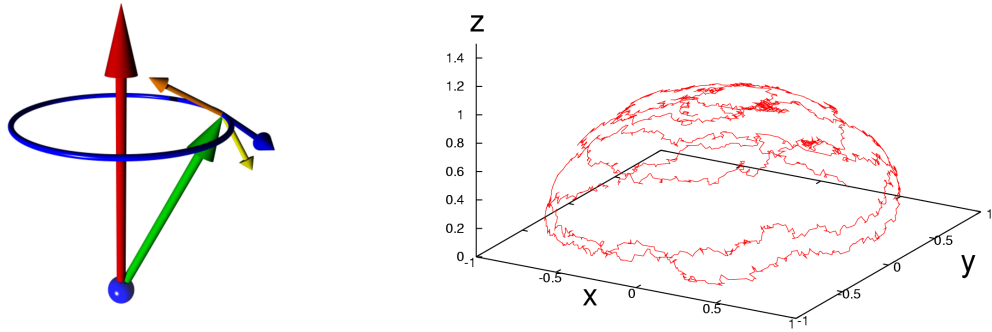


Abbildung 3: links: Stochastischer Term zur Simulation der thermischen Anregungen (gelb), rechts: Beispiel für Bewegungsverlauf eines Spins unter Einfluss von Dämpfung und Temperatur[2]

Bei der Gleichung (8) handelt es sich um eine stochastische Differentialgleichung. Gelöst wird sie im Falle dieses Programms mittels eines Runge-Kutta-Verfahrens 4. Ordnung für SDE mit schwachem Rauschen[5]. Die Genauigkeit der Lösung ist von der Ordnung $\mathcal{O}(\Delta t^4 + \epsilon^2 \Delta t^2)$. Die genaue Beschreibung des Lösungsweges findet sich im Anhang B.2.

3. OpenCL

OpenCL (**Open** Computing Language) ist eine Programmierplattform, die es dem Programmierer erlaubt, aufwändige Berechnungen auf andere Prozessoren, beispielsweise APUs (Accelerated Processing Unit) auszulagern. Diese ausgelagerten Programmteile werden *Kernel* genannt und müssen in der auf C-99 basierenden Programmiersprache OpenCL-C geschrieben sein. Der große Vorteil der OpenCL-Kernel ist, dass sie nicht für eine bestimmte Art von Prozessor geschrieben werden, sondern prozessorunabhängig sind. Sie werden erst zur Laufzeit des Hauptprogramms für eine spezielle Prozessorart kompiliert, egal ob für die Grafikkarte, eine andere CPU oder auch für auf GPU-Technik basierende Coprozessoren (z.B. Tesla). Dies ist ein Vorteil gegenüber dem Softwarepaket CUDATM der Firma NVIDIA, dessen Kernel nur auf NVIDIA-Karten ausführbar sind.

In der folgenden Grafik wird die Aufgabe deutlich, die OpenCL in einer Anwendung einnimmt:

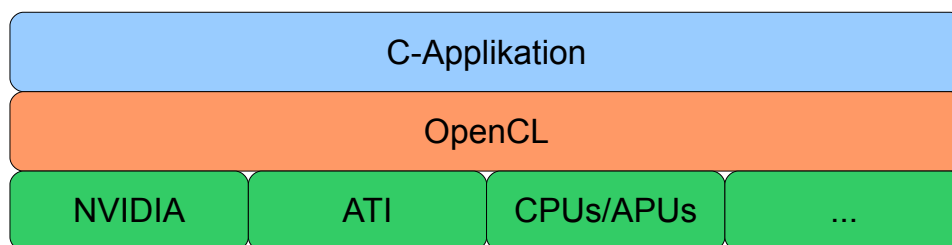


Abbildung 4: OpenCL als Abstraktionslayer

Wie schon erwähnt besteht ein OpenCL-Programm aus zwei Teilen, dem Hauptprogramm und dem Kernel. Inzwischen ist es in vielen Sprachen möglich, ein OpenCL-Hauptprogramm zu schreiben (C++, C, Fortran, Python u.a.). Dieser Hauptteil, welcher die OpenCL-Initialisierungen und Kernel-Aufrufe enthält, wird immer auf der CPU ausgeführt. Der oder die Kernel werden, nachdem sie im Hauptprogramm entsprechend kompiliert wurden, auf dem angegebenen Prozessor ausgeführt. Dieser Prozessor wird bei OpenCL *Device* genannt, die CPU ist der *Host*.

Die Arbeitsaufteilung bei der Ausführung des OpenCL-Kernels erfolgt über sogenannte Workgroups und Workitems. Ein Workitem kann im Kernel seine ID über die Funktion `get_global_id` ermitteln und davon ausgehend seine Berechnungen durchführen. Wird beispielsweise eine einfache Schleife parallelisiert, so berechnet jedes Workitem einen Schleifendurchlauf (siehe Codebeispiele 1 und 2).

```
int i;
for(i=0; i<numElements; i++)
{
    answer[i] = a[i] + b[i];
}
```

Listing 1: Schleife für Vektoraddition

Der entsprechende Kernel sähe wie folgt aus:

```
__kernel void add(__global float *a, __global float *b,
    __global float *answer, int numElements)
{
    int id = get_global_id(0);
    if(id < numElements){
        answer[id] = a[id] + b[id];
    }
}
```

Listing 2: Kernel für Vektoraddition

Auf Grund der Organisation der Workitems kann es dazu kommen, dass mehr Workitems existieren, als Berechnungen durchzuführen sind. Die Abfrage `id < numElements` ist daher notwendig, um fehlerhafte Speicherzugriffe zu vermeiden.

Die zu berechnenden Daten müssen in entsprechend angelegte Buffer auf dem Device geschrieben und die Buffer dem Kernel als Argumente mitgegeben werden. Nach der Ausführung des Kernels werden die berechneten Daten aus dem Ergebnisbuffer in den Hostspeicher kopiert.

Der generelle Ablauf eines OpenCL-Programms ist wie folgt:

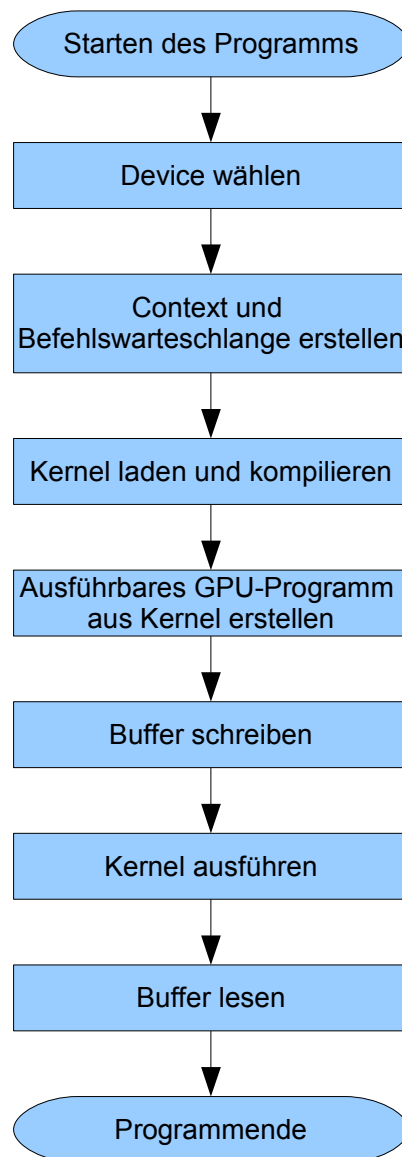


Abbildung 5: Ablauf eines OpenCL-Programms

Dies sind die für diese Arbeit wichtigsten Grundlagen und Eigenschaften von OpenCL. Eine ausführlichere Beschreibung von OpenCL findet sich in meiner Bachelorarbeit^[1] oder auf der offiziellen Webseite der Khronos Group^[6], welche den OpenCL-Standard definiert hat.

4. OpenGL

In vielen Bereichen ist Computergrafik ein wichtiges Thema, zum einen natürlich in der Computerspieleentwicklung, aber beispielsweise auch in der Wissenschaft, in der errechnete oder gemessene Daten durch Visualisierung deutlich anschaulicher werden. Da die Computergrafik so häufig und vielseitig eingesetzt wird, gibt es heutzutage einige sehr gute und ausgereifte Standards, die stetig weiterentwickelt und verbessert werden. Einer davon, der hier näher vorgestellt werden soll, ist OpenGL.

OpenGL (**Open Graphics Library**) ist eine API für interaktive 2D- oder 3D-Grafik. Der OpenGL-Standard wurde von einer Gruppe wichtiger Computerhersteller entwickelt, vorwiegend aus dem Bereich der Grafikprozessoren und -chips.

4.1. Grundlagen

Das Erstellen eines Bildes findet in OpenGL ursprünglich über eine sogenannte Grafik-Pipeline statt. Dieses Modell beschreibt, welche Schritte durchlaufen werden müssen, um eine 3D-Grafik auf dem Computermonitor darzustellen. Diese Funktionspipeline, auch Fixed Function Pipeline genannt, ist aber in vielerlei Hinsicht beschränkt und erlaubt nur wenige Anpassungen z.B. von Attributen oder Lichtquellen. Heutzutage ist es aufgrund der hochentwickelten Grafikkarten möglich, mit sogenannten selbstprogrammierten Shadern in die Pipeline einzugreifen und sie den eigenen Bedürfnissen anzupassen. Beispielsweise können Bearbeitungsschritte auf die GPU ausgelagert werden, die in der klassischen Pipeline auf der CPU stattfinden.

In dieser Arbeit werden zwar keine Shader benutzt, dafür aber die Nutzungsmöglichkeiten von sogenannten Vertex Buffer Objects (VBO), die es dem Programmierer ermöglichen, auf der CPU ausgeführte Zeichenfunktionen auf den Grafikkartenprozessor auszulagern, genau wie Shader das tun. Eine genauere Beschreibung der VBOs findet sich in Kapitel 4.4.

OpenGL an sich ist eine Bibliothek mit elementaren Funktionen zum Zeichnen von Szenen. Für die einfachere Einbettung in Anwendungen kann OpenGL über weitere Bibliotheken genutzt werden, welche beispielsweise das Verwalten der Fenster übernehmen. Die Abbildung 6 zeigt das Zusammenspiel von OpenGL und den anderen Bibliotheken, von denen GLUT[7] bzw. ihre Alternative GLFW[8] in den folgenden Kapiteln beschrieben werden.

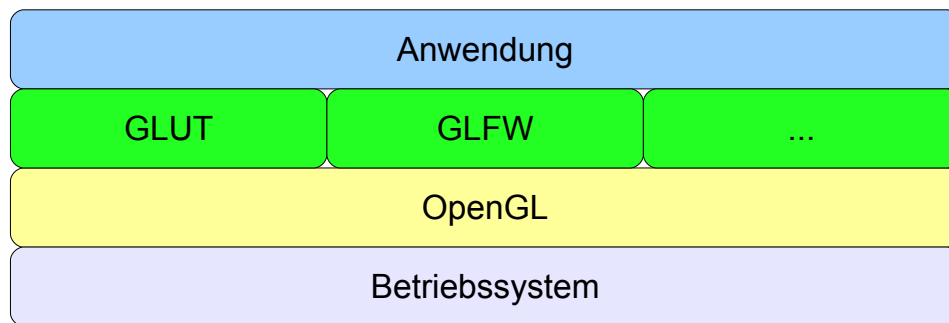


Abbildung 6: OpenGL als Abstraktionslayer

4.2. OpenGL Utility Toolkit

Häufig wird für die Darstellung des Zeichenfensters die Bibliothek GLUT (OpenGL Utility Toolkit) oder ihre freie Implementierung freeglut[9] verwendet. Zunächst werden die Parameter für die Fensterdarstellung gesetzt. Anschließend wird das Fenster geöffnet und die Visualisierung gestartet. Dies geschieht in GLUT durch den Aufruf der Funktion `glutMainLoop`, welche eine Endlosschleife startet. Die Funktion kehrt nie zum aufrufenden Programm zurück, ab diesem Zeitpunkt liegt die Verwaltung des Fensters allein in der Hand von GLUT. Der Programmierer kann nur über das Setzen von Callbackfunktionen in den Ablauf der Visualisierung eingreifen, wie zum Beispiel:

```
glutDisplayFunc(display);  
glutReshapeFunc(reshape);  
glutKeyboardFunc(onKbd);
```

Listing 3: Callbacks in GLUT

Die Parameter, die hier übergeben werden, sind Funktionen. `glutDisplayFunc` setzt die Funktion, die den Visualisierungsteil enthält, der also konkret etwas zeichnet. Mit `glutReshapeFunc` wird die Funktion festgelegt, die aufgerufen werden soll, wenn die Fenstergröße vom Benutzer verändert wird. Die Visualisierung wird von `reshape` an die neue Fenstergröße angepasst. Die Funktion `onKbd`, die durch `glutKeyboardFunc` gesetzt wird, enthält die Anweisungen für den Fall, dass der Benutzer eine Eingabe an der Tastatur macht. Die gedrückte Taste kann ausgelesen und darauf reagiert werden. Dies sind nur drei der möglichen Callbacks. Weitere sind beispielsweise `glutMouseFunc`, welche eine Funktion setzt, die auf Drücken der Maustasten reagiert, oder auch `glutIdleFunc`, eine sehr wichtige Funktion. Die Funktion, die sie als Parameter übergeben bekommt, wird immer dann aufgerufen, wenn die Mainloop zwischenzeitlich nicht mit Visualisierungen beschäftigt ist. Die Zeit kann zum Beispiel sinnvoll dafür verwendet werden, Berechnungen durchzuführen.

Üblicherweise ist es kein Problem, dass der Programmierer keine Möglichkeit des Eingreifens in die Mainloop von GLUT hat. In den meisten Programmen wird zunächst etwas berechnet und anschließend nur noch visualisiert. Problematisch wird es, wenn die Visualisierung nicht im Main-Programm abläuft, sondern durch einen Funktionsaufruf mit zeitaufwändigen Berechnungen gestartet werden soll.

4.3. GL framework

Eine Alternative zu GLUT, bei der dieses Problem nicht auftritt, ist die Bibliothek GLFW[8] (**GL Framework**). Es handelt sich um eine recht kleine Bibliothek, verglichen mit GLUT. Auch in dieser Bibliothek werden Callbacks gesetzt, ähnlich wie in GLUT:

```
glfwSetWindowCloseCallback(onClose);  
glfwSetWindowSizeCallback(onSize);  
glfwSetMouseButtonCallback(onMouseButton);
```

Listing 4: Callbacks in GLFW

Auch sonst ähneln sich GLUT und GLFW sehr. Der große Unterschied und Vorteil von GLFW ist, dass es ohne Mainloop auskommt. Der Programmierer kann die Zeichenschleife selbst implementieren:

```
while(isRunning && !glfwGetKey(GLFW_KEY_ESC))  
{  
    // do calculations  
    // call visualization routines  
}
```

Listing 5: Zeichenschleife in GLFW

So hat er die Möglichkeit, sie zu kontrollieren oder auch zu unterbrechen, ohne dass das Programm endet. In GLUT hingegen ist es nicht möglich, die Mainloop zu beenden, ohne dass das gesamte Programm beendet wird.

4.4. Vertex Buffer Objects

Üblicherweise werden für Daten (beispielsweise Vertex- oder Farbdaten) in OpenGL Arrays im Hauptspeicher angelegt. Diese werden bei der Ausführung des Programms in den Grafikkartenspeicher geladen, um sie visualisieren zu können. Seit OpenGL Version 1.5 werden aber auch sogenannte VBOs (**V**ertex **B**uffer **O**bjects) unterstützt. Vertex Buffer Objects sind spezielle Buffer, die es dem Programmierer erlauben, Daten direkt in den GPU-Speicher zu schreiben und diese dort auch zu manipulieren. Da so während der

Programmausführung ein Neuladen der Daten in den GPU-Speicher nach einer eventuellen Veränderung entfällt, beschleunigt die Nutzung der VBOs das Programm je nach Größe der Daten deutlich, denn aus technischen Gründen ist der Datenaustausch zwischen GPU und CPU der Punkt, an dem die schnelle GPU ausgebremst wird, wenn sie auf Daten aus dem Speicher des Rechners warten muss.

Wie VBOs in OpenGL angelegt und genutzt werden, wird im folgenden kurz anhand einiger Codefragmente erläutert.

Zunächst wird der Name des Buffers in Form einer eindeutigen ID angelegt. Über diese ID kann der Buffer später angesprochen werden.

```
GLuint id = 0;
glGenBuffers(1, &id);
```

Listing 6: Generieren der VBO-ID

Anschließend werden diese beiden Funktionen aufgerufen. Die erste, `glBindBuffer`, bindet den eben angelegten Namen des Buffers an eines der Targets `GL_ARRAY_BUFFER`, `GL_ELEMENT_ARRAY_BUFFER`, `GL_PIXEL_PACK_BUFFER` oder `GL_PIXEL_UNPACK_BUFFER`. Jede OpenGL-Aktion auf diesem Target, wie beispielsweise `glDrawArrays` (siehe Listing 9), arbeitet ab diesem Zeitpunkt auf dem Buffer. Die Funktion `glBufferData` erzeugt nun den Buffer mit einer Größe von *dataSize* Byte und initialisiert ihn mit den Daten, auf die der Pointer *data* zeigt. Mit *usage* liefert der Programmierer OpenGL einen Hinweis darauf, wie der Buffer verwendet werden soll. Mögliche Werte sind beispielsweise `GL_STATIC_READ`, was bedeutet, dass die Daten einmal geschrieben und dann viele Male gelesen werden, oder auch `GL_DYNAMIC_DRAW`, das aussagt, dass die Daten häufig von der Anwendung modifiziert und dann von OpenGL-Zeichenfunktionen verwendet werden.

```
glBindBuffer(target, id);
glBufferData(target, dataSize, data, usage);
```

Listing 7: Erzeugen des VBO

Als Veranschaulichung, wie VBO bei der Visualisierung verwendet werden, soll hier ein kleines Beispiel dienen. Folgender Code zeichnet *size* Punkte, wobei das Array *positions* die Position jedes einzelnen Punktes enthält. Es wurde vorher mit entsprechenden Werten initialisiert.

```
glBegin(GL_POINTS);
for( i=0;i<size*3;i+=3){
    // draw pixel as point
    glVertex3f(positions[i], positions[i+1], positions[i+2]);
}
```

```
glEnd();
```

Listing 8: Zeichnen ohne VBO

Bei Verwendung von VBO sieht der Code wie folgt aus:

```
glBindBuffer(GL_ARRAY_BUFFER, idpositions);
glVertexPointer(3, GL_FLOAT, 0, NULL);

//enable client state
glEnableClientState(GL_VERTEX_ARRAY);

glDrawArrays(GL_POINTS, 0, size);

//disable client state
glDisableClientState(GL_VERTEX_ARRAY);
```

Listing 9: Zeichnen mit VBO

Der Integer *idpositions* ist die ID des VBO, das wie oben beschrieben mit der Größe *size*3* angelegt wurde. Es wird zunächst an den `GL_ARRAY_BUFFER` gebunden, anschließend wird `glVertexPointer` aufgerufen, um festzulegen, dass es sich bei dem VBO um den Buffer für die Position handelt. Mit dem Aufruf von `glEnableClientState` wird es aktiviert. Nun können die Daten mit `glDrawArrays` gezeichnet werden. `GL_POINTS` gibt hier an, dass Punkte dargestellt werden sollen, und zwar *size* Stück, beginnend am Index 0 in den Buffern.

Das Ergebnis beider Codefragmente ist das gleiche, allerdings führt die Nutzung der VBO aus den oben genannten Gründen zu einer schnelleren Visualisierung. Dies ist vor allem bei großen Datenmengen und häufiger Aktualisierung der Grafik von Vorteil.

5. Fouriertransformation

Bei der Fouriertransformation handelt es sich um eine Methode, die ein periodisches Signal in sein Frequenzspektrum überführt. Ein periodisches Signal setzt sich aus vielen verschiedenen Frequenzen zusammen. Das Spektrum gibt den Anteil dieser Frequenzen im Signal an.

Angewendet werden solche Fouriertransformationen im Bereich der Signal- und Bildverarbeitung.

Die Formel für eine kontinuierliche Fouriertransformation sieht wie folgt aus:

$$F(\omega) = \int_{-\infty}^{\infty} f(t)e^{-i\omega t} dt \quad (9)$$

$f(t)$ ist das Signal. Als Ergebnis der Fouriertransformation erhält man eine komplexwertige Funktion $F(\omega)$, welche von den Frequenzen ω abhängt.

In der digitalen Signal- und Bildverarbeitung liegen Signale allerdings nie in kontinuierlicher Form vor, sondern immer diskret in einem endlichen Intervall. Außerhalb des Intervalls setzt sich das Signal periodisch fort. Um ein solches diskretes Signal in ein diskretes, periodisches Spektrum zu transformieren, wird die diskrete Fouriertransformation (DFT) verwendet. Sie ist eine Annäherung der kontinuierlichen FT durch der Rechteckregel. Die vorläufige Version der DFT sieht wie folgt aus:

$$F_d(v) = \sum_{n=0}^{N-1} f_n e^{-i2\pi v n T} \quad (10)$$

Die DFT liefert sinnvolle Werte im Bereich $[-\frac{1}{2T}, \frac{1}{2T}]$. Da aber für reellwertige Signalwerte f_i gilt, dass die Werte auf der negativen Achse konjugiert komplex zu den Werten auf der positiven Achse sind, wird die DFT nur im Bereich $[0, \frac{1}{2T}]$ ausgewertet^[10]. Die obere Intervallgrenze $\frac{1}{2T}$ wird auch als das Nyquist-Intervall bezeichnet.

Es gehen nur N unabhängige Werte in die DFT ein, nämlich die abgetasteten Signalwerte $f_0, \dots, f_{N-1}$. Es handelt sich bei der DFT, genau wie bei der kontinuierlichen FT, um eine komplexwertige Funktion, sie liefert also reelle und imaginäre Werte zurück. Da es nur N Eingangswerte gibt, kann es auch nur N unabhängige Ausgangswerte geben. Diese teilen sich auf in $N/2$ reelle und $N/2$ imaginäre Werte^[10]. Der reelle Anteil der Frequenz entspricht der Amplitude, der imaginäre der Phasenlage. Die DFT liefert also $N/2$ unabhängige Schwingungskomponenten, welche auf dem Intervall $[0, \frac{1}{2T}]$ liegen. Daraus ergeben sich die Auswertungsstellen $v_k = \frac{k}{NT}$, $k = 0, \dots, \frac{N}{2}$, wobei T das Abtastintervall

ist, mit welchem das Signal f_n aufgenommen wurde. Der Abstand zweier Frequenzen ist $\Delta v = \frac{1}{NT}$ und wird als spektrale Auflösung der Messung bezeichnet. Durch Einsetzen von v_k in die Gleichung ergibt sich folgende Definition der DFT:

$$F_d(v_k) = \sum_{n=0}^{N-1} f_n e^{-i2\pi kn/N} \quad (11)$$

Die Fast Fourier Transformation (FFT) ist ein sehr effizienter Algorithmus zur Berechnung einer DFT. Sie ist das grundlegende Verfahren zur Analyse von Signalen. Das Prinzip dahinter ist das so genannte „divide-and-conquer“ (teile und herrsche). Es werden Redundanzen in der Berechnung vermieden, dadurch wird zum einen das Verfahren deutlich beschleunigt, zum anderen erhält man genauere Ergebnisse durch Verminderung von Rundungsfehlern[10].

Wegen des „divide-and-conquer“-Prinzips ist eine wichtige Voraussetzung für die Anwendung der FFT, dass die Anzahl Abtaststellen eine Potenz von 2 ist ($2^n, n \in \mathbb{N}$). Dies schränkt zwar die Einsatzmöglichkeiten etwas ein, dafür ist die FFT aber sehr gut parallelisierbar, was im Rahmen dieser Masterarbeit ausgenutzt wurde.

6. Entwicklungsumgebung

Die CUDATM-basierte GPU-Anwendung, die im Rahmen dieser Masterarbeit umgeschrieben wurde, enthält Programmcode in verschiedenen Programmiersprachen. Der Initialisierungsteil besteht aus Fortran90-Code, die aufwändigeren Berechnungen hingegen sind in CUDATM und C realisiert und die Fouriertransformation sowie die grafische Darstellung der Ergebnisse befinden sich in Python-Skripts.

Im Laufe der Umgestaltung wurde der CUDATM-Teil durch OpenCL ersetzt und der Pythoncode wurde in OpenCL bzw OpenGL überführt. Damit sind für die neue Version des Spindynamics-Programms Compiler für folgende Programmiersprachen notwendig:

- C
- C++
- Fortran
- OpenCL
- OpenGL

Entwickelt bzw. umgeschrieben wurde die SD-Anwendung auf einem Rechner mit dem Betriebssystem Mac OS X 10.7 (Lion), welches das zur Zeit neuste Betriebssystem von Mac OS X der Firma Apple ist (Stand Juli 2012). Ab der Version Mac OS X 10.6 (Snow Leopard) werden die für OpenCL notwendigen Softwarepakete automatisch mitgeliefert. Unter Linux muss OpenCL manuell vom Programmierer installiert werden. Hier kann beispielsweise das CUDATM-Toolkits der Firma NVIDIA verwendet werden. Das Toolkit und die ebenfalls notwendigen *Developer Drivers* stellt NVIDIA auf seiner Homepage[\[11\]](#) zur Verfügung. Die aktuelle Version des CUDATM-Toolkit (Stand August 2012) ist die Version 5.0.

Das Paket, das AMD für seine ATI-Grafikkarten zur Verfügung stellt, ist das ATI Stream Software Development Kit[\[12\]](#).

Für C, C++ und Fortran wurden die entsprechenden Compiler (icc und ifort) von Intel verwendet.

In der neusten Version des Betriebssystems von Apple, Mac OS X 10.8 Mountain Lion, die im Juni 2012 veröffentlicht wurde, ist das LLVM-Compilerpaket (Low Level Virtual Machine) enthalten. Darin befindet sich das hoch performante Compiler-Frontend Clang[\[13\]](#), das die neusten Standards der Sprachen C, Objective-C und C++ unterstützt. Nach Migration des neuen Betriebssystems auf die Mac-Rechner des Instituts ist geplant, diesen Compiler zu verwenden.

7. Programmierung

7.1. OpenCL

Das Spindynamics-Programm lag in der Form eines Fortran-Programms vor, dessen parallele Abschnitte bereits in CUDATM realisiert waren. Im ersten Schritt wurde der CUDATM-Anteil in OpenCL umgeschrieben. Ziel dabei war, das Programm auch auf Rechnern ausführen zu können, die nicht über eine NVIDIA-Grafikkarte verfügen sondern beispielsweise über eine Grafikkarte der Firma AMD. Auf solchen Rechnern ist CUDATM nicht einsetzbar.

Im ursprünglichen Programm befanden sich die Kernel alle zusammen in der Datei „sdrk.cu“, zusammen mit dem Code zum Initialisieren von CUDATM und dem Aufrufen der Kernel, welcher mit der Direktive `extern "C" {...}` eingefügt war. Die Kernel wurden zunächst in einzelne Kernel-Files mit der Endung „.cl“ geschrieben, um sie, wie in OpenCL üblich, zur Laufzeit einlesen zu können. Kernel müssen bei OpenCL zur Laufzeit des Programms in Form von Strings vorliegen, aus denen anschließend die ausführbaren Kernel kompiliert werden. Alternativ hätten die Kernel auch direkt als Strings definiert werden können, da aber insgesamt zehn Kernel vorlagen, hätte dies das Programm sehr unübersichtlich gemacht.

Innerhalb der Kernel mussten selbstverständlich Änderungen vorgenommen werden, da sich die Syntax der Kernel-Programmiersprachen von CUDATM und OpenCL in einigen Punkten unterscheidet. So wird in CUDATM der Index eines Threads im Kernel wie folgt berechnet:

```
int idx = blockDim.x * blockIdx.x + threadIdx.x;
```

Listing 10: Thread-ID in CUDATM

In OpenCL spricht man nicht von Threads und Blöcken, sondern von Workitems und Workgroups. Das Prinzip ist aber sehr ähnlich. Die ID eines Workitems wird in einem OpenCL-Kernel über folgenden Aufruf abgefragt:

```
int id = get_global_id(0);
```

Listing 11: Workitem-ID in OpenCL

Neben der syntaktischen Anpassung mussten noch weitere Aspekte beachtet werden. So wurden beispielsweise aus den CUDATM-Kernen heraus Funktionen aufgerufen, die, wie die Kernel auch, in der Datei „sdrk.cu“ definiert sind. In OpenCL sind solche Funktionsaufrufe aus einem Kernel heraus nicht erlaubt, der Kernel hat keinen Zugriff auf diese Funktionen. Nur Funktionen, die in der OpenCL-Spezifikation implementiert sind, können vom Kernel aufgerufen werden, beispielsweise einige mathematische Funktionen.

Bei den aufgerufenen Funktionen handelte es sich allerdings nur um einfache Funktionen, beispielsweise solche zur Berechnung eines Skalarproduktes oder Kreuzproduktes von zwei `float3`-Variablen. Bei diesen wurde, da sie auch nicht allzu häufig verwendet wurden, ihr Aufruf im Kernel einfach durch den entsprechenden Code ersetzt. Andere kleine Funktionen wurden als Makros in die Kernel-Dateien eingefügt.

Auch Ausgaben im Kernel mittels `printf()` werden in der aktuellen OpenCL-Version (im Gegensatz zu CUDATM) nicht unterstützt. Dies erschwerte die Überführung des Programms in OpenCL, denn ein einfaches Debuggen der Kernel war so nicht möglich. Stattdessen wurden zwischenzeitlich Hilfsfelder an den Kernel übergeben, in welche die einzelnen Workitems ihre Zwischenergebnisse schrieben. Diese Methode war zwar etwas aufwändig, brachte aber trotzdem den gewünschten Erfolg, da so einige kleinere Berechnungsfehler gefunden werden konnten.

Die Aufrufe der Kernel mussten ebenfalls geändert werden. Sie sind in OpenCL deutlich aufwändiger als in CUDATM. Während sich in CUDATM ein Kernelaufruf in einer Zeile formulieren lässt, sind in OpenCL viele Vorbereitungen nötig. Vor dem ersten Ausführen des Kernels muss aus seinem Code ein Kernelprogramm erstellt werden. Die Argumente, die der Kernel benötigt, müssen einzeln gesetzt werden, erst dann kann der Kernel ausgeführt werden.

Zu Beginn der Arbeit trat ein Problem auf, das zunächst nicht nachvollziehbar war. Die OpenCL-Kernel liefen ohne Fehlermeldungen oder Warnungen durch, die errechneten Ergebnisse entsprachen aber in keinsten Weise den erwarteten. Der erste Verdacht war ein Berechnungsfehler in einem der Kernel. Da hier wie erwähnt ohne Testausgaben gearbeitet werden musste, gestaltete sich die Fehlersuche sehr zeitraubend. Es konnte jedoch in keinem der Kernel ein Fehler festgestellt werden. Schließlich wurde ein einfacher Testkernel geschrieben, um zu überprüfen, ob die Workitems und Workgroups korrekt angelegt waren und arbeiteten. Dabei wurde ein *float3*-Array an den Kernel übergeben, in welches jedes Workitem seine ID schreiben sollte. *float3* wurde verwendet, da auch die im SD-Programm relevanten Daten alle in *float3*-Arrays gespeichert waren. Dabei wurde festgestellt, dass bei der vorhandenen OpenCL-Version der Typ *float3* offensichtlich noch nicht korrekt unterstützt wird. Die Alternativen waren die Verwendung des Datentyps *float4* oder von *float*-Arrays. Da es aufwändiger erschien, die *float4*-Arrays in den Grafikkartenspeicher zu laden und in den Kernen entsprechend zu nutzen, wurde auf *float*-Arrays zurückgegriffen. Mit dieser Änderung lieferten die Berechnungen dann auch die erwarteten Ergebnisse.

Auf weitere Details der OpenCL-Programmierung soll hier nicht eingegangen werden, die wichtigsten Aspekte von OpenCL finden sich im Kapitel 3 (S. 11).

7.2. Visualisierung

Im ursprünglichen Programm wurden die fouriertransformierten Daten visualisiert. Diese Visualisierung fand mittels eines Python-Skriptes statt. Nach dem Einbau der OpenCL-FFT in das Programm werden die erzeugten Bilder in Dateien geschrieben (siehe Unterkapitel 7.3). Weitaus komplizierter und aufwändiger war die Visualisierung der Spindynamics-Daten. Die Visualisierung sollte zeitgleich mit der Berechnung der Spins stattfinden, nach jedem Schritt der parallelen Berechnung mittels OpenCL sollte die Grafik aktualisiert werden. Der einfache aber langsame Weg hierbei wäre es, nach jedem Berechnungsschritt die Spindaten aus dem GPU-Speicher auszulesen, an die Visualisierungsfunktionen zu übergeben und anschließend mit dem nächsten Berechnungsschritt fortzufahren. Schneller wird es bei Verwendung von OpenGL[14]. Da OpenGL seine Visualisierungsberechnungen ohnehin auf der GPU ausgeführt, ist ein Kopieren der Daten in den CPU-Speicher nicht notwendig. Für die Interoperation zwischen OpenCL und OpenGL gibt es Möglichkeiten, die im folgenden beschrieben werden.

7.2.1. VBO

Verwendet wurden zu diesem Zweck Vertex Buffer Objects (VBO)[15]. Es handelt sich hierbei um Buffer-Objekte, die direkt im GPU-Speicher angelegt werden und Vertex-Daten enthalten (s. Kapitel 4.4). Zunächst wurde ein neues C-Programm geschrieben (*visualization.c*). Es ist in der Grundform aufgebaut wie ein normales OpenGL-Programm in C. Allerdings hat es keine main-Funktion, da es beim Linken in das bestehende Programm eingebunden werden soll. Es besitzt stattdessen eine Funktion, die vom Hauptprogramm aus aufgerufen wird und die die nötigen OpenGL-Initialisierungen vornimmt. Auch die VBOs werden in diesem Zusammenhang erstellt und initialisiert.

Zum Zeichnen der Spins werden sowohl die Spindaten als auch die Positionen der einzelnen Atome benötigt. Es wird nur für die Spindaten ein VBO angelegt, da nur diese im Laufe des Programms immer wieder neu berechnet werden, die Positionen bleiben immer gleich. Wie Vertex Buffer Objects angelegt werden, wurde bereits im Grundlagenkapitel zu OpenGL beschrieben. Das Spin-VBO muss in jedem Berechnungsschritt von den OpenCL-Kernen gelesen und auch verändert werden. Dazu musste es im OpenCL-Teil des Programms bekannt gemacht werden. Ein VBO ist wie schon erwähnt eindeutig über seine ID identifizierbar. Diese musste daher von der Visualisierungsfunktion an der Fortran-Teil zurück- und dann an die OpenCL-Initialisierungsfunktion weitergegeben werden. Zunächst muss jedoch im OpenCL-Teil der Programms der aktuelle OpenCL-OpenGL-Kontext abgefragt und gespeichert werden:

```
//get current opengl-context
CGLContextObj glContext = CGLGetCurrentContext();
CGLShareGroupObj shareGroup = CGLGetShareGroup(glContext);
```

Listing 12: CL-GL-Kontext abfragen

Dieses so erzeugte `CGLShareGroupObj` muss beim Erzeugen des OpenCL-Kontextes als Attribut mitgegeben werden. Dies geschieht wie folgt:

```
cl_context_properties properties[] = {
    CL_CONTEXT_PROPERTY_USE_CGL_SHAREGROUP_APPLE,
    (cl_context_properties)shareGroup,
    0};
cxGPUContext = clCreateContext(properties, 1, &cdDevice,
    clLogMessagesToStderrAPPLE, NULL, &ciErrNum);
```

Listing 13: OpenCL-Kontext erzeugen

Hier muss zwischen den Betriebssystemen unterschieden werden. Die obigen Quellcode-Beispiele sind spezifisch für Mac OS X. Unter Linux muss der Kontext etwas anders angelegt werden. Die Unterscheidung zwischen den Betriebssystemen im Programm erfolgt über die Direktive `#ifdef __APPLE__`. Unter Linux wird folgender Code ausgeführt:

```
cl_context_properties properties[] = {
    CL_GL_CONTEXT_KHR, (cl_context_properties)
        glXGetCurrentContext(),
    CL_GLX_DISPLAY_KHR, (cl_context_properties)
        glXGetCurrentDisplay(),
    CL_CONTEXT_PLATFORM, (cl_context_properties)cpPlatform, 0};
cxGPUContext = clCreateContext(properties, 1, &cdDevice, 0,
    NULL, &ciErrNum);
```

Listing 14: OpenCL-Kontext unter Linux erzeugen

Nun sind OpenCL- und OpenGL-Kontext miteinander verknüpft. Abschließend muss nun noch der OpenCL-Buffer für die Spindaten angelegt werden. Das wird üblicherweise durch Aufruf der Funktionen `clCreateBuffer` und `clEnqueueWriteBuffer` gemacht:

```
cl_mem spins_d = clCreateBuffer(cxGPUContext,
    CL_MEM_READ_WRITE, (nspins)*3*sizeof(float), NULL,
    &ciErrNum);
ciErrNum |= clEnqueueWriteBuffer(cqCommandQueue, spins_d,
    CL_TRUE, 0, (nspins)*3*sizeof(float), (void *)spins, 0,
    NULL, NULL);
```

Listing 15: Erzeugen eines `cl_mem`-Objektes ohne VBO

Hierbei ist *spins_d* das zu erzeugende Bufferobjekt. Es wird zunächst mit der Größe $(nspins)*3*\text{sizeof}(\text{float})$ Bytes angelegt und anschließend mit den Werten aus dem float-Array *spins* belegt. Bei Verwendung des VBO erfolgt die Erzeugung und gleichzeitige Initialisierung durch die Funktion `clCreateFromGLBuffer`:

```
cl_mem spins_d = clCreateFromGLBuffer(cxGPUContext,
    CL_MEM_READ_WRITE, *idspinbuff, &ciErrNum);
```

Listing 16: Erzeugen eines `cl_mem`-Objektes mit VBO

Der Integer *idspinbuff* ist die ID des im OpenGL-Teil erzeugten VBO. Der OpenCL-Buffer wird hier nicht erneut angelegt, es wird nur ein `cl_mem`-Objekt mit dem VBO-Buffer als Speicherbereich erzeugt. Nun kann *spins_d* als Argument an den Kernel übergeben werden, so dass die Work-Items darauf zugreifen und ihre Berechnungen darauf ausführen können.

Da hier zwei Funktionen auf denselben Speicherbereich zugreifen, ist eine Synchronisation notwendig. Für diese Synchronisation muss der Programmierer sorgen, sonst könnte es passieren, dass beide Funktionen zeitgleich auf den Speicher zugreifen, der OpenCL-Kernel schreibend, die Visualisierung lesend. So kann es zu Fehlern bei der Visualisierung kommen. Noch fataler wäre es, würden beide Funktionen schreibend zugreifen.

Damit das nicht geschieht, wird der Zugriff auf das VBO wie folgt geregelt:

```
glFinish();
clEnqueueAcquireGLObjects(cqCommandQueue, 1, &spins_d, 0,
    NULL, NULL);
// launch kernel
// read back results
clFinish(cqCommandQueue);
clEnqueueReleaseGLObjects(cqCommandQueue, 1, &spins_d, 0,
    NULL, NULL);
```

Listing 17: Zugriffsregelung auf das VBO

Mit dem Aufruf von `glFinish` wird gewährleistet, dass alle Operationen der vorangegangenen OpenGL-Aufrufe abgeschlossen sind. Anschließend wird der Speicherbereich des VBO gesperrt und ausschließlich für OpenCL freigegeben. Somit kann keine OpenGL-Funktion mehr auf den Buffer zugreifen. Nun kann der Kernel ausgeführt werden, der die Daten im VBO verändert. Nach dem Zurücklesen der Daten wird zunächst `clFinish` aufgerufen, um zu warten, bis alle noch ausstehenden OpenCL-Aktionen beendet sind, und anschließend wird der Speicherbereich durch Aufruf von `clEnqueueReleaseGLObjects` wieder freigegeben.

Auf der Visualisierungsseite ist eine solche Synchronisation nicht notwendig. Dort wird nur lesend auf das VBO zugegriffen, gleichzeitiges Lesen von OpenCL und OpenGL verursacht keine Probleme.

Leider konnten die Vorteile des VBO in diesem Fall nicht komplett ausgenutzt werden. Das Problem war, dass die Spins, die im Bufferobjekt gespeichert und verändert werden, als Pfeile dargestellt werden sollten. Wie in Kapitel 4.4 - OpenGL im Codebeispiel 9 gezeigt, kann bei einfachen Grafikprimitiven wie Punkten oder Dreiecken das VBO direkt zum Zeichnen verwendet werden, wie in diesem Fall:

```
glBindBuffer(GL_ARRAY_BUFFER, idpositions);
glVertexPointer(3, GL_FLOAT, 0, NULL);
glDrawArrays(GL_POINTS, 0, size);
```

Listing 18: Zeichnen von Grafikprimitiven mit VBO

Hier befinden sich im VBO mit der ID *idpositions* die Vertexdaten von *size* Punkten, die durch den Aufruf von `glDrawArrays` alle direkt gezeichnet werden können.

Die Spins, deren Bewegung in diesem Programm berechnet wird, sollten allerdings als Pfeile gezeichnet werden. Pfeile sind in den Grafikprimitiven von OpenGL nicht vorhanden. Daher wurden bereits implementierte C++-Funktionen zum Zeichnen von Pfeilen (zusammengesetzt aus Zylindern und Kegeln) in C umgeschrieben und angepasst. Da für jeden Spin diese Zeichenfunktion aufgerufen werden musste, konnte die Funktion `glDrawArrays` nicht verwendet werden. Stattdessen mussten zunächst die Spindaten wieder aus dem VBO ausgelesen werden. Dies geschieht mit folgendem Funktionsaufruf:

```
glGetBufferSubData(GL_ARRAY_BUFFER, 0,
    size*3*sizeof(GLfloat), values);
```

Listing 19: Auslesen von Daten aus VBO

Durch den Aufruf von `glBindBuffer(GL_ARRAY_BUFFER, idspinbuff)` wurde das VBO mit den Spindaten ja zuvor an das Target *GL_ARRAY_BUFFER* gebunden. Darüber wird der Buffer nun identifiziert, um seine Daten auszulesen und in das Array *values* zu speichern. Dies geschieht zu Beginn der Zeichenfunktion. Anschließend werden die Spins in einer Schleife gezeichnet:

```
for(i=0; i<size*3; i+=3){
    drawArrow(positions[i], positions[i+1], positions[i+2],
        values[i], values[i+1], values[i+2]);
}
```

Listing 20: Zeichnen der Spins als Pfeile

Im Array *positions* befinden sich die Positionen der einzelnen Atome, in *values* die soeben aus dem VBO ausgelesenen Spins. Es wird an jede Position (`positions[i]..[i+2]`) der Vektor (`values[i]..[i+2]`) gezeichnet ($i=0,3,6,\dots,\text{size}\cdot 3$), wobei in den Zeichenfunktionen noch eine Anpassung der Länge gemacht wird.

Dieses Auslesen der Daten aus dem VBO und das anschließende Zeichnen in der Schleife nimmt leider etwas mehr Zeit in Anspruch als das direkte Zeichnen des VBOs. Die Zeichenfunktionen werden zwar noch auf der Grafikkarte ausgeführt, allerdings ist mehr Kommunikation erforderlich, da in jedem Schleifendurchlauf ein Aufruf der Zeichenfunktionen notwendig ist. Dies ist in diesem Fall leider unvermeidlich.

7.2.2. GLFW

Zu Beginn der Arbeit war geplant, mit GLUT (OpenGL Utility Toolkit) zu arbeiten. In dieser Bibliothek befinden sich viele OpenGL-Funktionen, hauptsächlich zum Zeichnen komplexerer Objekte und zur Kontrolle des Zeichenfensters. Bei der Nutzung von GLUT in diesem Programm ergab sich jedoch ein recht kompliziertes Problem. Das Öffnen des Fensters und das Zeichnen der Szene funktioniert nur über den Aufruf der GLUT-Funktion `glutMainLoop`. Diese Funktion kontrolliert über Callbacks die nötigen Methoden beispielsweise zum Neuzeichnen der Szene bei Verändern der Fenstergröße oder auch die Reaktionen auf Mausklicks oder Tastendruck. Eine Eigenart der Funktion ist, dass sie nie zum aufrufenden Programm zurückkehrt. Das ist bei den meisten OpenGL-Programmen kein Problem, da eine einmalige Berechnung einer Szene mit anschließender Visualisierung vorgenommen wird. Die Initialisierungen und Berechnungen finden üblicherweise vorher statt. Im Falle der SD-Anwendung gibt aber ein Fortran Hauptprogramm den Ablauf vor, initialisiert die Daten und ruft die Berechnungen sowie die Visualisierung auf.

Die erste Idee dieses Problem zu umgehen waren Threads. Es sollte ein zusätzlicher Thread gestartet werden, in welchem die Glutmainloop laufen sollte. Es stellte sich jedoch heraus, dass dieses Verfahren sehr betriebssystemabhängig ist. Unter Mac OS X beispielsweise darf die Glutmainloop nicht aus einem Thread heraus aufgerufen werden, dies führt zu Fehlern im Programm. Da aber klar war, dass die Visualisierung von einer Funktion aufgerufen und aktualisiert werden musste, wurde nach Alternativen zu GLUT gesucht. Nach einigen Recherchen wurde die Bibliothek GLFW gewählt, da diese nicht ereignisgesteuert arbeitet und dem Programmierer ermöglicht wird, den Ablauf zu bestimmen (siehe Kapitel 4.3).

Leider konnte der Plan, die Visualisierung und der OpenCL-Teil des Programms in unterschiedlichen Threads laufen zu lassen, nicht umgesetzt werden. Die Verwendung von Threads sollte das Programm beschleunigen, da die OpenCL-Kernel schon die neuen Berechnungen hätten durchführen können, während die alten Daten zeitgleich visualisiert worden wären. Das erste kleinere Problem war das Erstellen von Threads in Fortran. Die

üblichen Thread-Bibliotheken wie POSIX Threads[16] sind nicht für Fortran definiert. Nach einigen Tests und Recherchen wurde allerdings ein POSIX-Thread-Interface für Fortran gefunden[17]. Folgendes Codebeispiel aus dem Interface zeigt, wie die pthread-Funktionsaufrufe für Fortran bekannt gemacht werden:

```
#define CRETRD cretrd_  
void CRETRD(void *(*thread_func)(void *), pthread_t  
    *theThread) {  
    pthread_create(theThread, NULL, thread_func, NULL);  
}
```

Listing 21: Definieren einer für Fortran aufrufbaren Funktion

Diese Funktion kann nun aus Fortran heraus aufgerufen werden, wobei *visualize_f* eine Fortran-Routine ist und *tid* ein Integer, auf welchem die ID des erstellten Threads gespeichert wird:

```
call CRETRD(visualize_f,tid)
```

Listing 22: Aufruf aus Fortran

Tests zeigten, dass es mit dem genannten Interface tatsächlich möglich war, verschiedene Threads aus Fortran heraus zu erzeugen. Es musste lediglich ein kleiner Trick angewendet werden, um auch C-Funktionen durch die Threads ausführen zu lassen. Eine direkte Übergabe des C-Funktionsnamens in **CRETRD** war nicht möglich. Daher wurden kleine Hilfsroutinen in folgender Form geschrieben:

```
subroutine visualize_f()  
    call initdata(npp,Spinmatrix,x,y,z,idspinbuff)  
    call visualize()  
end subroutine visualize_f
```

Listing 23: Umweg über Fortran-Routine

So konnten auch mehrere C-Funktionen aus einem Thread heraus aufgerufen werden. Theoretisch funktionierte also alles, was nötig war, um OpenCL und OpenGL parallel in zwei unterschiedlichen Threads laufen zu lassen. Dann allerdings trat ein unerwartetes Problem auf, die Funktion **CGLGetCurrentContext** lieferte **NULL** zurück. Bisher hatte dieser Aufruf problemlos funktioniert. Nach weiteren Recherchen stellte sich heraus, dass erst ab OpenCL-Version 1.1 die Interaktion zwischen OpenCL und OpenGL in verschiedenen Threads unterstützt wird. Auf der Grafikkarte des Rechners, auf dem das Programm geschrieben und getestet wurde, war jedoch nur OpenCL-Version 1.0 verfügbar. Da nicht davon ausgegangen werden kann, dass auf jedem Rechner die entsprechende OpenCL-Version vorhanden ist, wurde diese Idee nicht weiter verfolgt, sie kann aber eventuell später umgesetzt werden (siehe Kapitel 9 Ausblick).

7.2.3. Benutzerinteraktion

Eine Vorgabe für die Funktionalitäten des Programms war, dass der Benutzer die Möglichkeit haben sollte, die erzeugte Grafik seinen Bedürfnissen entsprechend anzupassen. Es wurden zunächst die wichtigsten Funktionen implementiert, ein späteres Hinzufügen von weiteren Optionen ist aber problemlos möglich (siehe Kapitel 9 Ausblick (S. 45)).

Implementiert wurde unter anderem die Möglichkeit, in die Grafik hinein- und wieder herauszuzoomen. Um die Bedienung möglichst intuitiv zu gestalten, wurden diese Aktionen an die Tasten $+$ und $-$ geknüpft. Im Code sieht diese Aktion wie folgt aus:

```
float scale=1.;

static
void onSize(int w, int h)
{
    [...]
    /* Map abstract coords directly to window coords. */
    glOrtho(halfx-scale*rangex/2, halfx+scale*rangex/2,
            halfy-scale*rangey/2, halfy+scale*rangey/2, -10, 10);
    [...]
}

static
void onKbd(int key, int action)
{
    unsigned char k = (unsigned char) key;
    switch(k){
        [...]
        case '+':
            {
                scale-=0.05;
            }break;
        case '-':
            {
                scale+=0.05;
            }break;
    }
    int w,h;
    glfwGetWindowSize(&w,&h);
    onSize(w,h);
}
```

Listing 24: Zoomen in der Grafik

Es wird die globale Variable *scale* verwendet, um den aktuellen Zoomfaktor zu speichern.

`onSize` ist die Callbackfunktion, die aufgerufen wird, wenn das Fenster in seiner Größe verändert wird. Hier wird unter anderem die Funktion `glOrtho` aufgerufen, welche die abstrakten Koordinaten der Objekte durch eine Parallelprojektion auf die Fensterkoordinaten abbildet. Die übergebenen Argumente geben die Ausmaße des Sichtvolumens an.

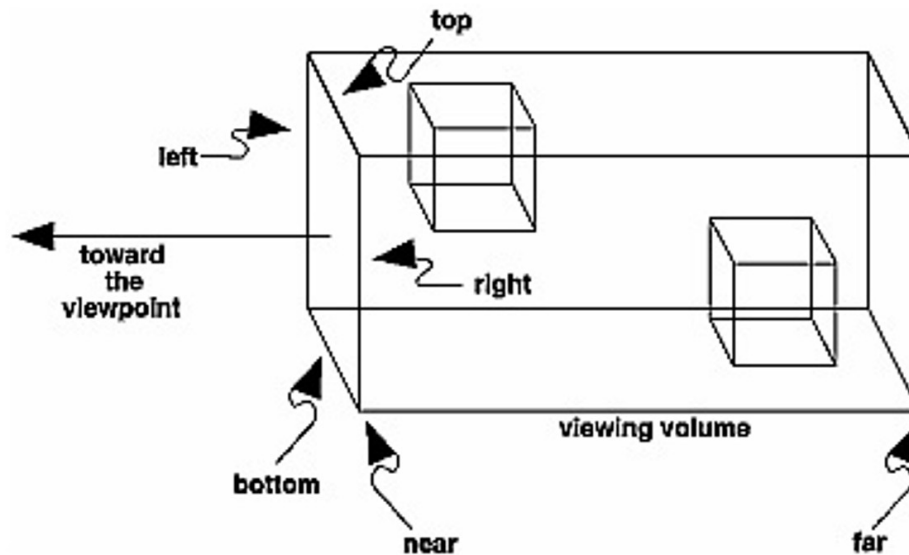


Abbildung 7: Orthogonalprojektion[18]

Im Funktionsaufruf werden die sechs Parameter wie folgt übergeben:

```
glOrtho(left, right, bottom, top, near, far);
```

Listing 25: Argumente von `glOrtho`

Die Variable *scale* verkleinert bzw vergrößert das Sichtvolumen also, wenn ihr Wert verändert wird. Der Aufruf der Callbackfunktion `onSize` sorgt hier dafür, dass das Fenster sofort nach Ändern des Wertes neu gezeichnet wird.

7.3. FFT

Nach der Durchführung aller Iterationsschritte sollte eine 2D-Fouriertransformation auf die Spindaten angewendet werden. Im ursprünglichen Programm wurde die FFT mittels eines Python-Skriptes durchgeführt. Die Anforderung an die Umsetzung der FFT war, dass sie in OpenCL implementiert sein sollte, um eine möglichst schnelle Berechnung zu gewährleisten.

Es wäre ein unnötiger Mehraufwand gewesen, die FFT selbst zu implementieren, da es bereits fertige Implementierungen in OpenCL gibt. Nach einigen Recherchen wurde eine Implementierung der Apple Inc.[\[19\]](#) gewählt. Es handelt sich um ein C++-Programm, in dem die FFT in einem OpenCL-Kernel ausgeführt wird. Es waren einige Mac OS X spezifische `#include`-Direktiven enthalten, die für die Nutzung des Programms unter Linux angepasst werden mussten. Dies geschah wie im folgenden Codebeispiel:

```
#ifndef __APPLE__
#include <OpenCL/OpenCL.h>
#else
#include <CL/opencl.h>
#endif
#include <stdio.h>
```

Listing 26: Unterscheidung bei Einbindung der Header-Dateien

Die aktuelle Version der Programms ermöglicht das Berechnen von 1D-, 2D- und 3D-Fouriertransformationen, sowohl vorwärts als auch invers. In diesem Fall wurde nur eine einfache 2D-FFT benötigt. Es handelt sich bei der Implementierung um ein Programm, in welchem an einem Beispieldatensatz die verschiedenen FFT-Versionen durchgeführt und die Ergebnisse auf Korrektheit überprüft werden. Voraussetzung für die Verwendung einer FFT ist, dass die Anzahl der Abtastpunkte, also in diesem Fall die Anzahl der Spins in x- und y-Richtung, eine Potenz von 2 ist, also die Form 2^n hat. Auf diesen Punkt wird später genauer eingegangen.

Folgende relevante Dateien sind in der Implementierung enthalten:

- `clFFT.h`
- `fft_base_kernels.h`
- `fft_internal.h`
- `procs.h`
- `fft_execute.cpp`
- `fft_kernelstring.cpp`
- `fft_setup.cpp`
- `main.cpp`

- Makefile
- param.txt
- ReadMe.txt

Zunächst war geplant, nur den Kernelcode aus dem Programm zu übernehmen. Nach Einarbeiten in das Programm war jedoch klar, dass der Kernel erst während der Ausführung des Programms generiert wird, und dass neben dem Kernel auch viele Hilfsfunktionen benötigt werden. So wurden aus `main.cpp` des FFT-Programms die benötigten Funktionsaufrufe in die Funktion `fft_opengl_` in `sdrk.c` übernommen.

Bevor eine 2D-FFT auf die Spinmatrix angewendet werden kann, müssen die Daten zunächst umsortiert werden. Die Matrix muss sich für die Anwendung einer Fouriertransformation in x- sowie in y-Richtung periodisch fortsetzen. Dies geschieht mit Hilfe einer Basismatrix, die angibt, welcher Spin an welche Stelle gehört. Die Basismatrix wird ganz zu Beginn des Programms initialisiert und mit Werten belegt. Im Code sieht die Umsortierung wie folgt aus:

```
do i=1,basismatrixd1
  do j=1,basismatrixd2
    l=basismatrix(i,j,4)
    basismatrix(i,j,1)=data2(1,l)
    basismatrix(i,j,2)=data2(2,l)
    basismatrix(i,j,3)=data2(3,l)
  enddo
enddo
allocate( datR(N1,N2) )
datR=basismatrix(:,:,3)
```

Listing 27: Umsortieren der Daten

In `data2` befinden sich die Spins nach dem letzten Iterationsschritt, `basismatrix(i,j,4)` beinhaltet die Nummer des Spins, der in der neuen Matrix an Position (i,j) stehen soll. Nach der Umsortierung wird `datR` angelegt, eine Matrix mit den selben Dimensionen wie die Basismatrix. Durch die Zuweisung von `basismatrix(:,:,3)` werden nur die z-Werte der Spins in `datR` gespeichert, es wird also später eine FFT der z-Komponenten durchgeführt. Für eine FFT in x- oder y-Richtung müsste entsprechend eine Zuweisung von `basismatrix(:,:,1)` bzw. `basismatrix(:,:,2)` stattfinden.

Nach der Umorganisierung der Spindaten und Zuweisung auf `datR` kann die 2D-FFT angewendet werden. Wie bereits angesprochen unterstützt die FFT nur Datengrößen der Form 2^n . Da dies natürlich nicht immer gegeben ist, wurde für die anderen Fälle eine in Fortran implementierte DFT in das Programm integriert. Die Unterscheidung zwischen der OpenCL-FFT und der Fortran-DFT ist wie folgt implementiert:


```

M1=ceiling(log2(N1*1.0))
M2=ceiling(log2(N2*1.0))
if (M1-log2(N1*1.0)==0 .and. M2-log2(N2*1.0)==0) then
    write(*,*) 'Do FFT on GPU'
    call fft_openc1(datR,datFFT,N1,N2)
else
    write(*,*) 'Do DFT on CPU'
    call DFT2D(datR,datI,N1,N2,datFFT)
end if

```

Listing 28: Unterscheidung zwischen OpenCL-FFT und Fortran-DFT

Da vorgegeben war, dass nur reellwertige Daten transformiert werden müssen, wird dem Aufruf von `fft_openc1` kein Array für die komplexen Anteile mitgegeben.

Die Visualisierung der FFT-Daten erfolgt zum aktuellen Zeitpunkt nicht über OpenGL, sondern über die Ausgabe in eine Bilddatei im .ppm-Format (Portable Pixmap)[20]. Dieses Dateiformat lässt sich beispielsweise mit OpenOffice Draw[21] oder dem Bildbearbeitungsprogramm gimp[22] öffnen. Bei Problemen, die sich schneller berechnen lassen, wäre eine Live-Visualisierung durchaus möglich, vor allem auch wegen der Nutzung der VBOs, die die Kommunikation zwischen dem Berechnungsteil des Programms und der Visualisierung minimieren. Die Berechnung der FFT dauert dafür jedoch zu lang, deshalb ist sie für eine Live-Visualisierung nicht geeignet.

Die FFT wird daher erst am Ende des Programms berechnet, und es genügt die Ausgabe der so erzeugten Bilder in .ppm-Dateien. Die Daten werden nach jedem der 16 Schritte der Berechnungen in einem Array gespeichert. Ein Schritt besteht hier aus 250.000 Iterationen, wobei bei jedem Schritt eine andere Temperatur angenommen wird, um so das Verhalten der Spins bei unterschiedlichen Temperaturen untersuchen zu können. Jede der 16 Datensätze wird am Ende fouriertransformiert, zur Zeit nur jeweils die z-Komponente.

Theoretisch ist es auch möglich, den Benutzer des Programms wählen zu lassen, welchen der Schritte und welche der drei Komponenten er fouriertransformieren möchte. Diese Wahl ist bereits implementiert.

7.4. Kompilierung und Ausführung

Um das Spindynamics-Programm ausführen zu können, muss es zunächst kompiliert werden. Es wurde ein Makefile geschrieben, um dies für den Benutzer so einfach wie möglich zu gestalten:

```
OBJS = SD.o sdrk.o visualization.o fthread.o
FFTOBJS = fft/fft_setup.o fft/fft_kernelstring.o
         fft/fft_execute.o
HEADERS = fft/procs.h fft/fft_internal.h
         fft/fft_base_kernels.h fft/clFFT.h
INCLUDES = -I/usr/local/cuda/include

FC = ifort
FFLAGS = -O -g -fPIC
CFLAGS = -g -fPIC -Wall
CC = icc
CXX = icc
CXXFLAGS = $(CFLAGS)
LD = ifort
LDFLAGS = -g

default:
    make 'uname'
Linux:
    make sd LDFLAGS="-Wl,-rpath,/usr/local/lib64" \
    LIBS="-lOpenCL -lglfw -lGLEW -lXrandr -lrt -lstdc++"
Darwin:
    make sd LIBS="-framework OpenCL -framework OpenGL -lGLEW
    -lglfw -lstdc++ -framework Cocoa -framework IOKit
    -framework Accelerate -framework AppKit"

%.o: %.f90
    $(FC) $(FFLAGS) -o $@ -c $<

%.o: %.c
    $(CC) $(CFLAGS) $(INCLUDES) -o $@ -c $<

%.o: %.cpp
    $(CXX) $(CXXFLAGS) $(INCLUDES) -o $@ -c $<

sd: $(OBJS) $(FFTOBJS)
    $(LD) $(LDFLAGS) -o $@ $(OBJS) $(FFTOBJS) $(LIBS)

SD.o: sdesolver/sde_rungekutta7.f90

clean:
    rm -f *.o sd
    rm -f fft/*.o
```

Listing 29: Makefile

Durch den Aufruf **make** in der Konsole im Programmverzeichnis wird das Makefile ausgeführt. Es erkennt automatisch, ob es sich beim Betriebssystem um Mac OS X oder um ein Linuxsystem handelt. Die Kompilierung und Ausführung des Programms auf einem Windows-Rechner wird zum aktuellen Zeitpunkt noch nicht unterstützt.

Das Makefile kompiliert das Programm entsprechend dem erkannten Betriebssystem. Es erzeugt eine ausführbare Datei mit dem Namen *sd*. Diese kann in der Konsole über **./sd** gestartet werden. Die Spindaten sowie die Positionen der Moleküle des Atoms werden aus den Dateien *inp.txt*, *in_plist.in* sowie *in_mulist.in* eingelesen.

Nach der Initialisierung der Daten werden die Bewegungen der Spins auf der GPU berechnet und gleichzeitig visualisiert. Im Visualisierungsfenster hat der Benutzer die Möglichkeit, durch Tastatureingaben einige Änderungen durchzuführen. Folgende Möglichkeiten stehen ihm bei der aktuellen Version des Programms zur Verfügung:

Taste	Ereignis
c	Setzen der Farbe des Spins anhand seiner x, y oder z-Komponente oder einfarbig
+	Hineinzoomen in die Grafik
–	Herauszoomen aus der Grafik
ESC	Schließen des Visualisierungsfensters und Beenden des Programms

7.5. Programmüberblick

Einen groben Überblick über den Programmablauf des umgeschriebenen Programms liefert folgendes Sequenzdiagramm:

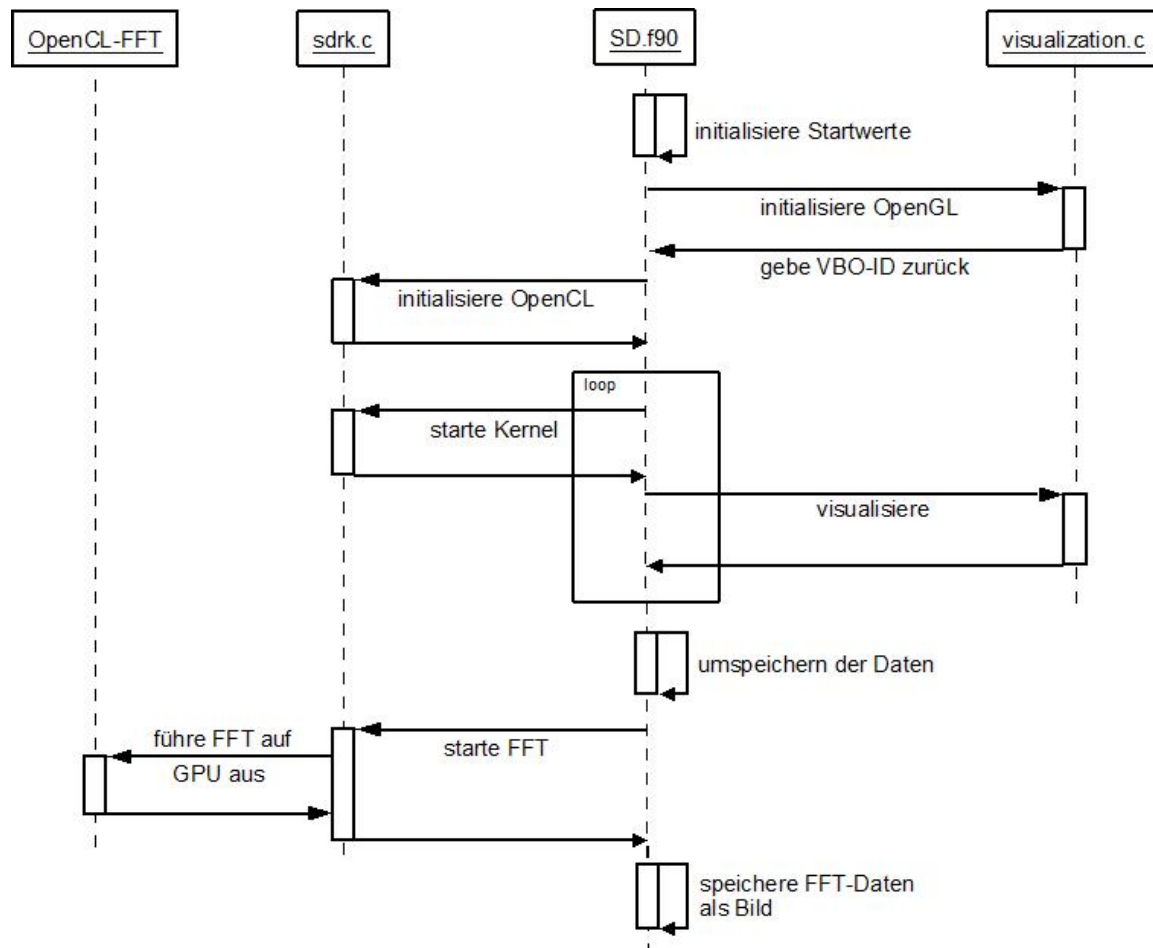


Abbildung 8: Paralleles Programm mit Visualisierung

8. Ergebnisse und Zusammenfassung

Eines der wichtigsten Kriterien bei der Bewertung eines OpenCL-Programms ist seine Laufzeitverbesserung gegenüber dem seriellen Programm auf der CPU. Daher wurden in das Spindynamics-Programm Zeitmessungen eingebaut. Getestet wurde das ursprüngliche Programm auf einer handelsüblichen CPU. Es handelte sich hierbei um einen *Intel Core2 Duo* Prozessor mit 3,06 GHz. Das neue Programm wurde auf einer schwächeren *NVIDIA GeForce 8800 GS* GPU mit 384 MB Speicher und 64 Cores und auf einer sehr leistungsstarken Grafikkarte ausgeführt, einer *NVIDIA GeForce GTX 590* mit 3.072 MB Speicher und 1024 Cores (siehe auch Anhang A).

Die Programme wurden mit einem Datensatz getestet, der aus 6000 Spins besteht. Berechnet wurden jeweils 16 Schritte mit jeweils 250.000 Iterationen. Vor jedem Schritt wird die Temperatur verändert. Es wurde die Zeit gemessen, die die Programme für 250 Iterationen benötigen, was einem Prozent eines Schritts der Berechnungen entspricht. Folgende Zeiten ergaben sich dabei:

Prozessor	Intel Core2 Duo	GeForce 8800 GS	GeForce GTX 590
<i>Laufzeit</i>			
Berechnungen	3.9 s	11.7 s	0.3 s
Visualisierung	-	5.3 s	-

Die Tabelle zeigt, dass bei Verwendung der leistungsstarken GPU ein Beschleunigungsfaktor von mehr als 10 bei der Berechnung erreicht wurde. Auf die Ausführungszeit des gesamten Programms hochgerechnet ergibt sich ein Verhältnis von etwas mehr als 100 Minuten beim ursprünglichen Programm zu etwa 9 Minuten bei der OpenCL-Version. Gerade wenn mehrere Datensätze getestet werden sollen, ist dies ein enormer Zeitgewinn.

Auffällig ist, dass bei der schwächeren Grafikkarte die Berechnung durch Nutzung von OpenCL deutlich verlangsamt wurde. Der Grund hierfür ist, dass die GPU nicht schnell genug ist, um die anfallende Kommunikationszeit mit der CPU auszugleichen. Hier würde es eventuell helfen, nicht jede Berechnung in einen Kernel auszulagern, sondern nur die rechenintensivsten, um die Kommunikation zu reduzieren (siehe Kapitel 9 Ausblick).

Leider war es nicht möglich, auf der *NVIDIA GeForce GTX 590* eine Zeitmessung der Visualisierung durchzuführen. Es ist aber zu erwarten, dass das Verhältnis von Berechnungs- zu Visualisierungszeit ähnlich ist wie auf der *NVIDIA GeForce 8800 GS*. Die genaue Dauer der Visualisierung ist hier aber auch nicht von Bedeutung, es sollte in dieser Arbeit vor allem eine Möglichkeit aufgezeigt werden, wie eine OpenGL-Visualisierung in ein OpenCL-Programm integriert werden kann.

Eine vergleichende Zeitmessung von OpenCL-FFT und der DFT in Fortran wurde noch nicht eingebaut. Da die Fouriertransformation erst am Ende durchgeführt wird, spielt ihre Laufzeit keine so große Rolle. Ihr Anteil an der Gesamtlaufzeit des Programms ist selbst bei 16 durchgeführten DFTs noch minimal.

Um die Entwicklung der Spins im Laufe der gesamten Berechnung zu beobachten, wurden die Daten nach jedem der 16 Schritte gespeichert, um am Ende des Programms auf diesen Daten jeweils eine 2D-Fouriertransformation (hier in z-Richtung) durchzuführen. Repräsentativ sollen hier vier der erzeugten Bilder gezeigt werden, um die Entwicklung zu verdeutlichen:

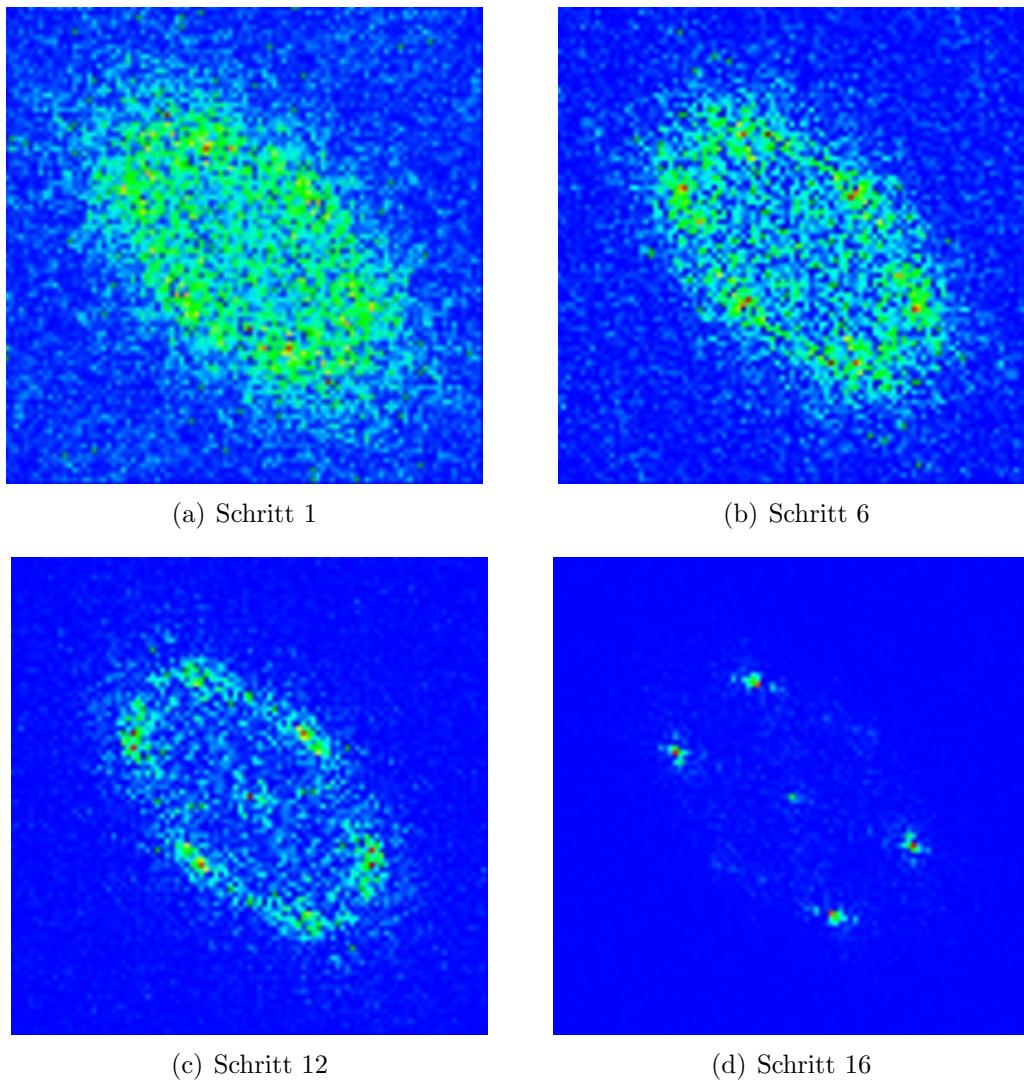


Abbildung 9: Spektren der Spindaten nach verschiedenen Schritten

Zusammenfassend lässt sich sagen, dass es im Laufe dieser Masterarbeit mehr Probleme und Hindernisse gab, als zunächst vermutet. Dass die Kombination von Threads und der OpenGL-OpenCL-Interaktion solche Schwierigkeiten bereiten würde, war zu Beginn der Arbeit nicht abzusehen. Wäre dieses Problem nicht aufgetreten, hätte man das Programm um etwa ein Drittel (siehe obige Tabelle der Zeitmessungen) beschleunigen können, da mit der Ausführung der Kernel nicht hätte gewartet werden müssen, bis die Visualisierung aktualisiert wurde.

Die Nutzung eines VBO hingegen war, auch wenn seine Vorteile nicht komplett ausgenutzt werden konnten, sehr hilfreich für die Verknüpfung von Berechnung und Visualisierung. Die Daten hätten sonst bei jedem Visualisierungsaufruf übergeben werden müssen. Sollten neuere Versionen von OpenCL oder OpenGL die Nutzung von Threads in ihrer Interaktion unterstützen, wird die Verwendung des VBO unerlässlich. In diesem Fall wird es keine wiederholten Aufrufe der Visualisierung mehr geben, sie wird nur einmal zu Beginn des Threads gestartet und aktualisiert sich bei Änderung der Daten selbst. Auch die Synchronisation des Speicherzugriffs wird hier wichtig werden. Da die Berechnungs- und die Visualisierungsfunktion zum aktuellen Zeitpunkt nacheinander aufgerufen werden, ist ein gleichzeitiger Speicherzugriff nicht möglich und eine Synchronisation im Grunde überflüssig. Sie wurde im Hinblick auf die eventuelle Nutzung von Threads trotzdem eingebaut, um späteren Problemen vorzubeugen.

9. Ausblick

In einigen Punkten kann das Spindynamics-Programm noch erweitert und verändert werden, beispielsweise um die Ausführungszeit zu verringern oder um es benutzerfreundlicher zu machen.

Vor allem bei der Visualisierung sind noch einige Erweiterungen vorzusehen. Zeitlich hätte es den Rahmen dieser Masterarbeit überschritten, beim jetzigen Stand des Programms sollte es aber keine große Schwierigkeiten bereiten. Der Benutzer des Programms sollte neben den schon implementierten Aktionen zusätzlich noch folgende Möglichkeiten haben, die Grafik zu manipulieren:

- Farbe des Spins anhand eines Einheitsvektors e in eine beliebige Richtung setzen ($\text{color} = \text{spin} * e$)
- Darstellung der Spins alternativ als Kegel statt als Pfeile
- Frei rotierbare Kamera (gesteuert durch Maus oder Tastatur)
- Bei 3D-Objekten Schnitte durch die Struktur, so dass nur die Spins in dieser Ebene angezeigt werden

All diese Aktionen können als Reaktion auf Mausklicks oder Tastatureingaben in die vorhandenen Callback-Funktionen `onMouseButton` und `onKbd` eingebaut werden. Am sinnvollsten ist hier vermutlich die Nutzung globaler Variablen sowie deren Veränderung in den Callbacks, so wie dies bei den vorhandenen Aktionen implementiert wurde (siehe Listing 24 auf Seite 33).

Interessant wäre eventuell auch eine Berechnung und Live-Visualisierung der FFT-Daten nach jedem Berechnungsschritt der Spindaten. Dies bietet sich aber vermutlich nur für Datensätze der Größe 2^n an, da hier die schnellere OpenCL-FFT verwendet werden kann. Die Visualisierung müsste in dem Fall in OpenGL implementiert werden, ähnlich wie die Visualisierung der Spins. Hier könnten sogar die Vorteile der VBOs vollständig ausgenutzt werden, da die FFT-Daten lediglich als farbige Punkte dargestellt werden, nicht als Pfeile wie die Spins. Daher würde die Visualisierung das Programm vermutlich nur geringfügig langsamer machen.

Beim aktuellen Stand des Programms werden alle Berechnungen zu den Spinbewegungen in OpenCL-Kernen ausgeführt. Eventuell wird das Programm noch etwas schneller, wenn nur der rechenintensivste Teil, das Runge-Kutta-Verfahren, auf der GPU ausgeführt wird, die Vorberechnungen aber wie ursprünglich auf der CPU stattfinden. Denn einige Kernel enthalten nur recht simple Berechnungen, so dass die benötigte Kommunikation zur Ausführung des Kerns mehr Zeit in Anspruch nimmt, als durch die Parallelisierung der Berechnung gewonnen wird. Es könnten Testmessungen durchgeführt

werden, um die optimale Ausführungszeit zu bestimmen.

Die hier eingesetzten Techniken können zudem als Vorlage für weitere Simulationsprogramme dienen. Idealerweise könnte man die Programmbausteine als generisch verwendbares API weiterentwickeln. Benötigt würden vor allem die Dimensionen und die Größe der zu visualisierenden Daten, sowie die gewünschte Art der Darstellung. Das API müsste lediglich die IDs der erzeugten VBOs zurückgeben. Mit den IDs wäre der Programmierer in der Lage, OpenCL-Memoryobjekte zu erzeugen und diese an seine Kernel zu übergeben.

A. Prozessordaten

Intel Core2 Duo CPU

OpenCL Version: OpenCL C 1.1
Compute Units: 2
Max Work Item Dimensions: 3
Max Workitem Sizes: 1024 / 1 / 1
Max Work Group Size: 1024

NVIDIA GeForce 8800 GS

OpenCL Version: OpenCL 1.0
Compute Units: 8
Max Work Item Dimensions: 3
Max Workitem Sizes: 512 / 512 / 64
Max Work Group Size: 512

NVIDIA GeForce GTX 590

OpenCL Version: OpenCL C 1.1
Compute Units: 16
Max Work Item Dimensions: 3
Max Workitem Sizes: 1024 / 1024 / 64
Max Work Group Size: 1024

B. Lösung der SDE

Die beiden folgenden Unterkapitel sind aus der Diplomarbeit „Atomistic Spin-Dynamics in Confined Magnetic Nano-Structure“^[2] von David Bauer übernommen. Sie beschreiben die Lösung einer SDE mittels eines Runge-Kutta-Verfahrens. Die Fettschreibung einer Variablen zeichnet sie hier als Vektor aus ($\mathbf{S}$).

B.1. Translation of the stochastic differential equation

In the following section the numerical algorithm used in this thesis for solving the following stochastic differential equation with multiplicative white noise will be discussed.

$$\frac{\partial \mathbf{S}_i}{\partial t} = \mathbf{h}_i \times \mathbf{S}_i - \lambda(\mathbf{h}_i \times \mathbf{S}_i) \times \mathbf{S}_i + \mathbf{f}_i(t) \times \mathbf{S}, \quad \mathbf{h}_i = \frac{\partial \mathcal{H}}{\partial \mathbf{S}_i} \quad (12)$$

The Hamiltonian $\mathcal{H}$ depends on the N different spin vectors $\{\mathbf{S}\} = \{\mathbf{S}_1, \mathbf{S}_2, \dots, \mathbf{S}_N\}$ and the random variables $\mathbf{f}_i(t)$ are given by a distribution of white noise. First, it is useful to rewrite the stochastic differential equation to an expression (see equation (13)), which is commonly used for the numerical analysis of stochastic differential equations. The differential equation is in the Ito interpretation of the integral and, therefore, needs to be transformed into the Stratonovich type, which is explained in section 2.4.2 (^[2] Seite 24):

$$d\mathbf{S} = \mathbf{a}(\{\mathbf{S}\}) + \epsilon \sum_{r=1}^{3N} \boldsymbol{\sigma}_r(\{\mathbf{S}\}) \circ dW_r \quad (13)$$

$$= \mathbf{a}(\{\mathbf{S}\}) + \epsilon^2 \mathbf{b}(\{\mathbf{S}\}) dt + \epsilon \sum_{r=1}^{3N} \boldsymbol{\sigma}_r(\{\mathbf{S}\}) dW_r \quad (14)$$

For $\mathbf{a}$, $\mathbf{b}$ and $\boldsymbol{\sigma}_r$ the following expressions will be used:

$$a_i^\alpha = -A_i^\alpha - \lambda C_i^\alpha, \quad \mathbf{A}_i = \mathbf{s}_i \times \mathbf{h}_i, \quad \mathbf{C}_i = \mathbf{s}_i \times (\mathbf{s}_i \times \mathbf{h}_i)$$

$$b_i^\alpha(\mathbf{s}) = \frac{1}{2} \sum_{r=1}^{3N} \sum_{j=1}^N \sum_{\beta=x,y,z} \frac{\partial \sigma_r^{(i,\alpha)}(\mathbf{s})}{\partial s_j^\beta} \sigma_r^{(i,\beta)}(\mathbf{s})$$

$$\begin{aligned} \sigma_{3i-1}^{(i,x)} &= s_i^z, & \sigma_{3i}^{(i,x)} &= -s_i^y, & \sigma_{3i}^{(i,y)} &= s_i^x \\ \sigma_{3i-2}^{(i,y)} &= -s_i^z, & \sigma_{3i-2}^{(i,z)} &= s_i^y, & \sigma_{3i-1}^{(i,z)} &= -s_i^x \end{aligned}$$

The term $\mathbf{a}$ represents the driving force of the Hamiltonian, $\mathbf{b}$ is an expression which appears by transforming the Ito interpretation of the integral to the Stratonovich and $\boldsymbol{\sigma}_r$ is a sparse matrix which represents the crossproduct of the fluctuating term in equation (12).

B.2. Runge-Kutta Integration Scheme

Equation (14) is numerically solved by a weak Runge-Kutta (4th order) method for stochastic differential equations with small noise developed by Milstein and Tretyakov [5]. The method has a weak global error of $\mathcal{O}(h^4 + \epsilon^2 h^2)$ where $h = t_i - t_{i-1}$ denotes the timestep. By applying the Runge-Kutta algorithm to the equation of motion (14) one ends up with [4]:

$$\begin{aligned}
 \mathbf{S}(t_{k+1}) = & \mathbf{S}_k + \epsilon h^{1/2} \sum_{r=1}^{3N} \boldsymbol{\omega}_r \xi_r + \epsilon^2 h \sum_{j,r=1}^{3N} \mathbf{c}_{j,r} \xi_{j,r} \\
 & + \frac{\epsilon h^{3/2}}{2} \sum_{r=1}^{3N} \mathbf{d}_r \xi_r + \frac{1}{6} [\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4] \\
 & + \frac{\epsilon^2 h}{2} \left[\mathbf{b} \cdot \mathbf{S}(t_k) + b \left(\mathbf{S}(t_k) + \epsilon h^{1/2} \sum_{r=1}^{3N} \boldsymbol{\omega}_r \xi_r \right. \right. \\
 & \left. \left. + h[a(\mathbf{S}(t_k)) + \epsilon^2 \mathbf{b}(\mathbf{S}(t_k))] \right) \right]
 \end{aligned} \tag{15}$$

The function $\mathbf{b}(\mathbf{s})$ is in our case simply given by $\mathbf{b}(\mathbf{s}) = -\mathbf{s}$ and the none zero components of $\mathbf{c}_{jr}$ and $\mathbf{d}_r$ are given by:

$$\begin{aligned}
 c_{3i-2,3i-1}^{(i,x)} &= S_i^y(t_k) \quad , \quad c_{3i-2,3i}^{(i,x)} = S_i^z(t_k) \\
 c_{3i-1,3i-1}^{(i,x)} &= -S_i^x(t_k) \quad , \quad c_{3i,3i}^{(i,x)} = -S_i^x(t_k) \\
 c_{3i-1,3i-2}^{(i,y)} &= S_i^x(t_k) \quad , \quad c_{3i-1,3i}^{(i,y)} = S_i^z(t_k) \\
 c_{3i-2,3i-2}^{(i,y)} &= -S_i^y(t_k) \quad , \quad c_{3i,3i}^{(i,y)} = -S_i^y(t_k) \\
 c_{3i,3i-2}^{(i,z)} &= S_i^x(t_k) \quad , \quad c_{3i,3i-1}^{(i,z)} = S_i^y(t_k) \\
 c_{3i-2,3i-2}^{(i,z)} &= -S_i^z(t_k) \quad , \quad c_{3i-1,3i-1}^{(i,z)} = -S_i^z(t_k)
 \end{aligned} \tag{16}$$

$$\begin{aligned}
 d_{3i-1}^{(i,x)} &= a_i^z(\mathbf{S}(t_k)) + \epsilon^2 b_i^z(\mathbf{S}(t_k)) \\
 d_{3i}^{(i,x)} &= a_i^y(\mathbf{S}(t_k)) + \epsilon^2 b_i^y(\mathbf{S}(t_k)) \\
 d_{3i}^{(i,y)} &= a_i^x(\mathbf{S}(t_k)) + \epsilon^2 b_i^x(\mathbf{S}(t_k)) \\
 d_{3i-2}^{(i,y)} &= a_i^z(\mathbf{S}(t_k)) + \epsilon^2 b_i^z(\mathbf{S}(t_k)) \\
 d_{3i-2}^{(i,z)} &= a_i^y(\mathbf{S}(t_k)) + \epsilon^2 b_i^y(\mathbf{S}(t_k)) \\
 d_{3i-1}^{(i,z)} &= a_i^x(\mathbf{S}(t_k)) + \epsilon^2 b_i^x(\mathbf{S}(t_k)),
 \end{aligned} \tag{17}$$

The random variables ξ_r and ζ_r are generated at each timestep according to the following stochastic distribution:

$$P(\xi = 0) = 2/3, \quad P(\xi = \sqrt{3}) = 1/6, \quad P(\xi = -\sqrt{3}) = 1/6 \tag{18}$$

$$P(\zeta = -1) = 1/2, \quad P(\zeta = 1) = 1/2$$

$$\xi_{jr} = 1/2(\xi_j \xi_r - \gamma_{jr} \zeta_j \zeta_r), \quad \gamma_{jr} = \begin{cases} -1, & j < r \\ +1, & j \geq r \end{cases}$$

$P(\xi = a)$ describes the probability of an event $\xi = a$. The stochastic distribution is (i.e. (18)) formed as follows: Three partitions are created which are assigned to a random event ($[0, 1/6] \rightarrow 1, [1/6, 1/3] \rightarrow -1, [1/3, 1] \rightarrow 0$). A random number generator which creates a uniform distribution in the range $[0, 1]$ is used. Depending on in which partition the random number occurs the different random events are assigned.

Literatur

- [1] Anna Jakobs
Parallelisierung einer mesoskopischen Simulationmethode (MPC) auf der Basis von OpenCL (Open Computing Language)
August 2010
- [2] David Bauer
Atomistic Spin-Dynamics in Confined Magnetic Nano-Structure
Oktober 2008
- [3] J. Weber
Fluctuation Dissipation Theorem
29. September 1955
- [4] V. P. Antropov, S. V. Tretyakov and B. N. Harmon
Spin dynamics in magnets: Quantum effects and numerical simulations
1997
- [5] G. N. Milstein und M. V. Tret'yakov
Numerical methods in the weak sense for stochastic differential equations with small noise
Dezember 1997
- [6] Khronos Group
OpenCL
<http://www.khronos.org/opengl>
(Zugriff: Juni 2012)
- [7] Khronos Group
GLUT - The OpenGL Utility Toolkit
<http://www.opengl.org/resources/libraries/glut/>
(Zugriff: Juli 2012)
- [8] elmindreda
GLFW Reference Manual
<http://www.glfw.org/GLFWReference27.pdf>
(Zugriff: Juli 2012)
- [9] The freeglut Programming Consortium
The Free OpenGL Utility Toolkit
<http://freeglut.sourceforge.net/> (Zugriff: Juli 2012)
- [10] Prof. Dr. rer. nat. Walter Hillen
Digitale Signalverarbeitung
Oktober 2010

- [11] NVIDIA Corporation
CUDA Toolkit
<http://www.nvidia.com/content/cuda/cuda-toolkit.html>
(Zugriff: August 2012)
- [12] Advanced Micro Devices, Inc.
ATI Stream Software Development Kit (SDK) v1.4-beta
<http://developer.amd.com/archive/gpu/ATISStreamSDKv1.4Beta/Pages/default.aspx>
(Zugriff: August 2012)
- [13] Apple Inc.
OS X Mountain Lion Core Technologies Overview
http://movies.apple.com/media/us/osx/2012/docs/OSX_MountainLion_Core_Technologies_Overview.pdf
Juni 2012
- [14] Khronos Group
OpenGL
<http://www.khronos.org/opengl/> (Zugriff: Juni 2012)
- [15] Khronos Group
Vertex Buffer Object
http://www.opengl.org/wiki/Vertex_Buffer_Object
(Zugriff: Juli 2012)
- [16] Blaise Barney, Lawrence Livermore National Laboratory
POSIX Threads Programming
<https://computing.llnl.gov/tutorials/pthreads/>
(Zugriff: Juli 2012)
- [17] V. Ganesh, Research Student, University of Pune, India
Using Pthreads in Fortran
<http://v-ganesh.tripod.com/papers/fthreads.pdf>
(Zugriff: Juli 2012)
- [18] Dr. H. Zilken
Programmieren mit OpenGL Teil 3
<http://www2.fz-juelich.de/vislab/vorlesung/download/praktikum/p3.pdf>
(Zugriff: Juli 2012)
- [19] Apple Inc.
OpenCL_FFT
http://developer.apple.com/library/mac/#samplecode/OpenCL_FFT/Introduction/Intro.html
(Zugriff: Juli 2012)

- [20] Bryan Henderson
PPM Format Specification
<http://netpbm.sourceforge.net/doc/ppm.html>
(Zugriff: Juli 2012)
- [21] The Apache Software Foundation
OpenOffice.org 3.x - Draw
<http://www.openoffice.org/de/product/draw.html>
(Zugriff: Juli 2012)
- [22] The GIMP Team
GIMP - The GNU Image Manipulation Program
<http://www.gimp.org/>
(Zugriff: Juli 2012)
- [23] Wilhelm Homberg
OpenCL Basics
28.05.2010
- [24] Wilhelm Erkens, Dietmar Koschmieder
Programmierung in Fortran 90/95
1999
- [25] NVIDIA Corporation
NVIDIA OpenCL JumpStart Guide Version 0.9
April 2009
- [26] Advanced Micro Devices, Inc.
Porting CUDA Applications to OpenCL
<http://developer.amd.com/zones/openclzone/programming/pages/portingcudatoopencl.aspx>
(Zugriff: Mai 2012)
- [27] ECLIPSE ENGINEERING, LLC
Understanding OpenCL-OpenGL Interoperability
<http://www.dyn-lab.com/articles/cl-gl.html>
(Zugriff: Juli 2012)
- [28] Advanced Micro Devices, Inc.
OpenCLTM and the AMD APP SDK
<http://developer.amd.com/documentation/articles/pages/OpenCL-and-the-AMD-APP-SDK.aspx>
(Zugriff: Juli 2012)

Abbildungsverzeichnis

1.	Bewegung des Spins (grün) um einen Vektor des effektiven Feldes (rot)[2]	8
2.	links: Dämpfung der Spinbewegung durch Dämpfungsterm (orange), rechts: Beispiel für Bewegungsverlauf eines gedämpften Spins[2]	9
3.	links: Stochastischer Term zur Simulation der thermischen Anregungen (gelb), rechts: Beispiel für Bewegungsverlauf eines Spins unter Einfluss von Dämpfung und Temperatur[2]	10
4.	OpenCL als Abstraktionslayer	11
5.	Ablauf eines OpenCL-Programms	13
6.	OpenGL als Abstraktionslayer	16
7.	Orthogonalprojektion[18]	34
8.	Paralleles Programm mit Visualisierung	40
9.	Spektren der Spindaten nach verschiedenen Schritten	42

Listings

1.	Schleife für Vektoraddition	12
2.	Kernel für Vektoraddition	12
3.	Callbacks in GLUT	16
4.	Callbacks in GLFW	17
5.	Zeichenschleife in GLFW	17
6.	Generieren der VBO-ID	18
7.	Erzeugen des VBO	18
8.	Zeichnen ohne VBO	18
9.	Zeichnen mit VBO	19
10.	Thread-ID in CUDA TM	25
11.	Workitem-ID in OpenCL	25
12.	CL-GL-Kontext abfragen	28
13.	OpenCL-Kontext erzeugen	28
14.	OpenCL-Kontext unter Linux erzeugen	28
15.	Erzeugen eines cl_mem-Objektes ohne VBO	28
16.	Erzeugen eines cl_mem-Objektes mit VBO	29
17.	Zugriffsregelung auf das VBO	29
18.	Zeichnen von Grafikprimitiven mit VBO	30
19.	Auslesen von Daten aus VBO	30
20.	Zeichnen der Spins als Pfeile	30
21.	Definieren einer für Fortran aufrufbaren Funktion	32
22.	Aufruf aus Fortran	32
23.	Umweg über Fortran-Routine	32
24.	Zoomen in der Grafik	33
25.	Argumente von glOrtho	34
26.	Unterscheidung bei Einbindung der Header-Dateien	35
27.	Umsortieren der Daten	36
28.	Unterscheidung zwischen OpenCL-FFT und Fortran-DFT	37
29.	Makefile	38